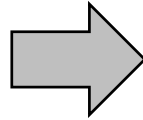


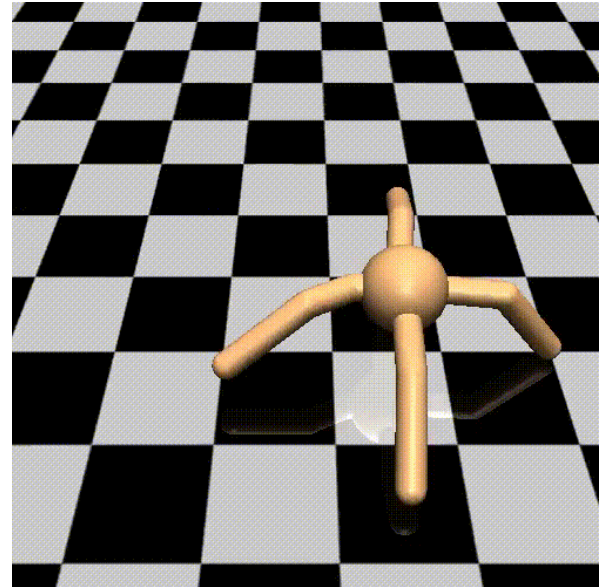
Introduction to Deep Reinforcement Learning

Deep Reinforcement Learning (Deep RL)

Deep Learning



Deep RL



- **What is it?** Framework for learning to solve sequential decision making problems.
- **How?** Trial and error in a world that provides occasional rewards
- **Deep?** Deep RL = RL + Neural Networks

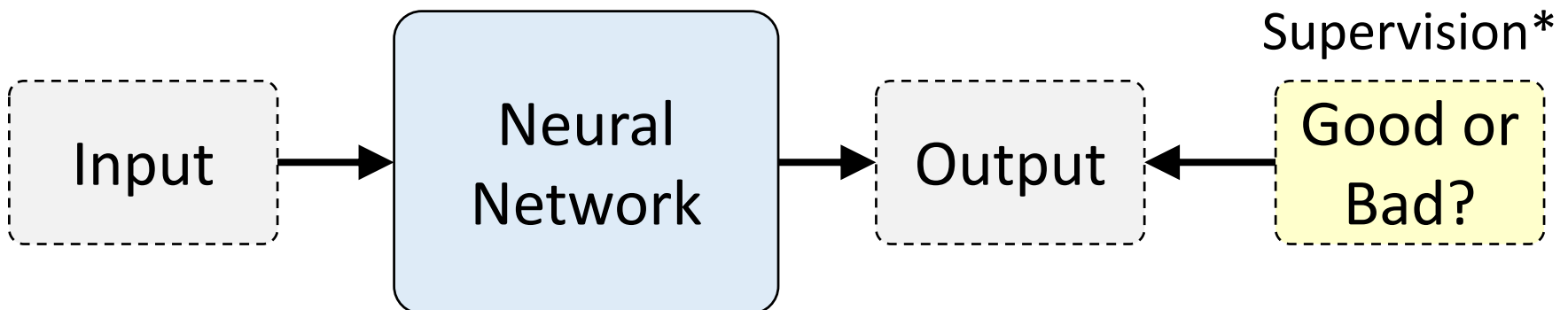
Types of Learning

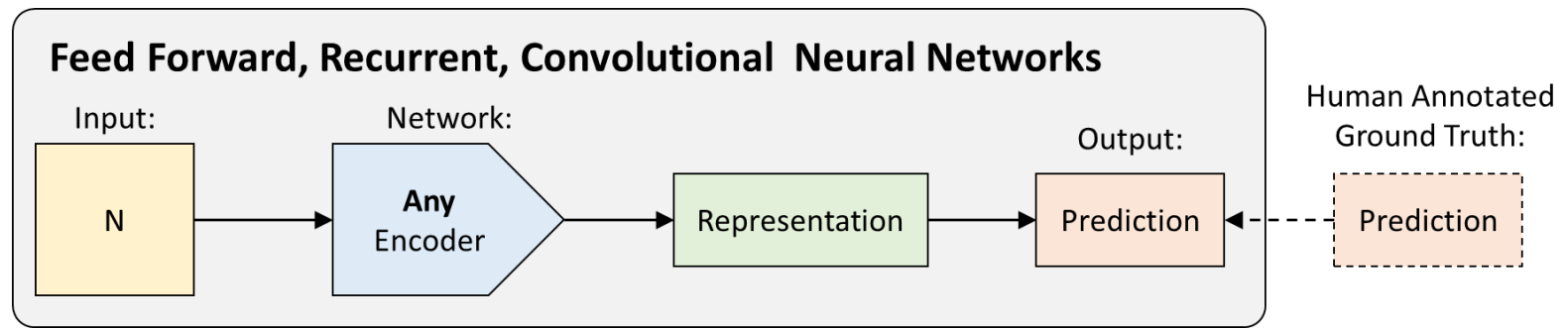
- Supervised Learning
- Semi-Supervised Learning
- Unsupervised Learning
- Reinforcement Learning



It's all “supervised” by a loss function!

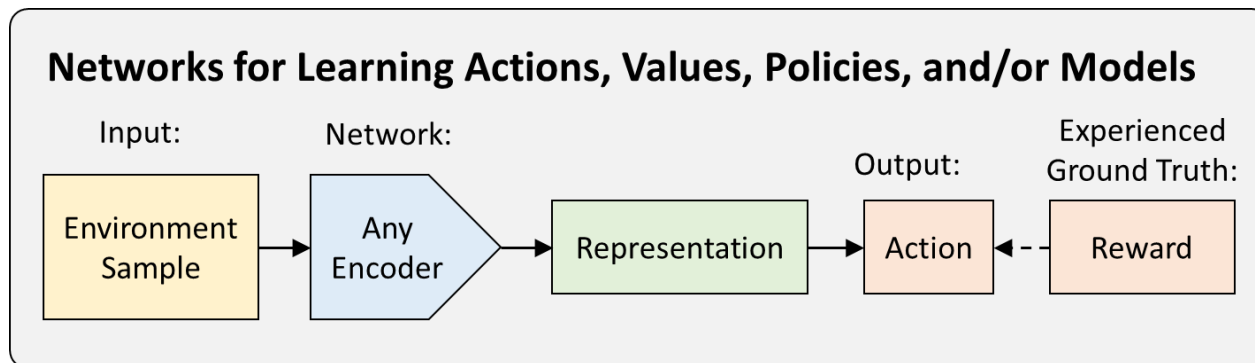
**Someone has to say what's good and what's bad (see Socrates, Epictetus, Kant, Nietzsche, etc.)*





Supervised learning is “teach by **example**”:
Here’s some examples, now learn patterns in these example.

Reinforcement learning is “teach by **experience**”:
Here’s a world, now learn patterns by exploring it.



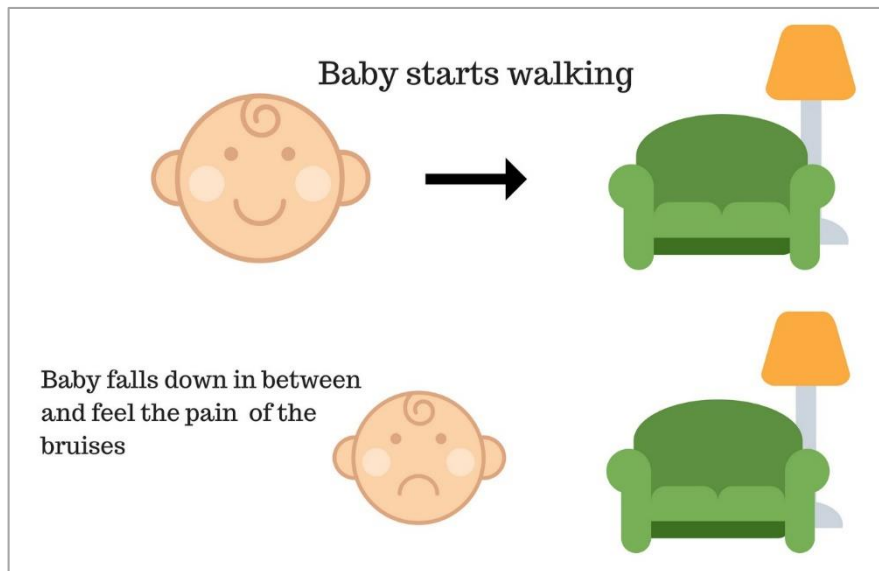
Machine Learning: Supervised vs Reinforcement

Supervised learning is “teach by **example**”:

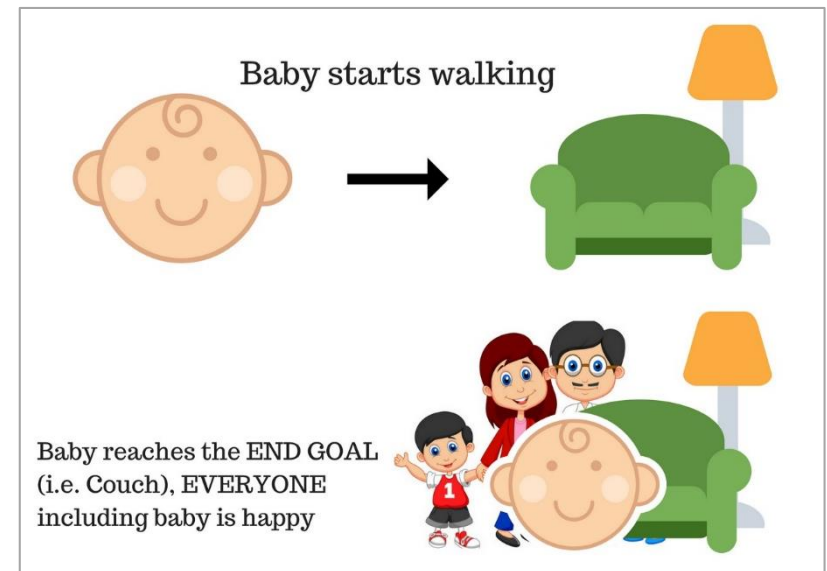
Here’s some examples, now learn patterns in these example.

Reinforcement learning is “teach by **experience**”:

Here’s a world, now learn patterns by exploring it.



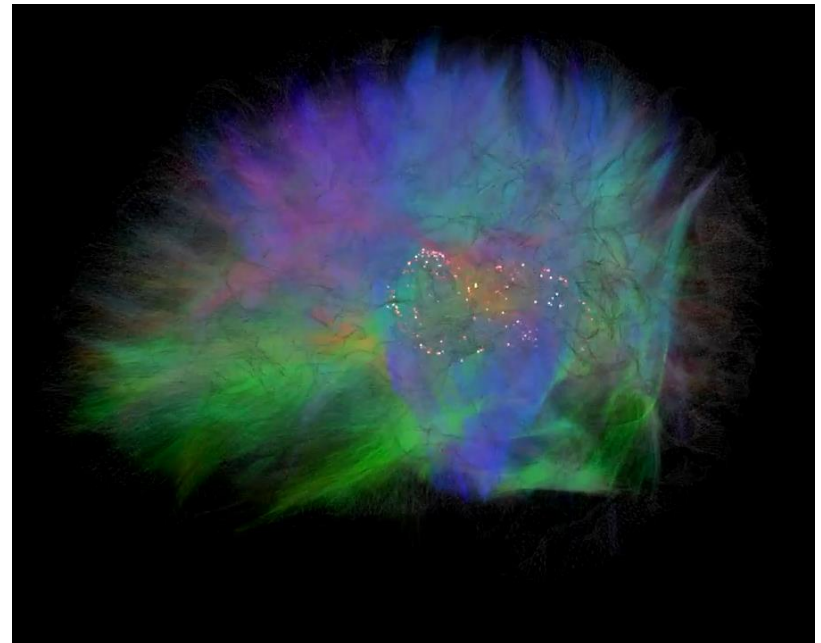
Failure



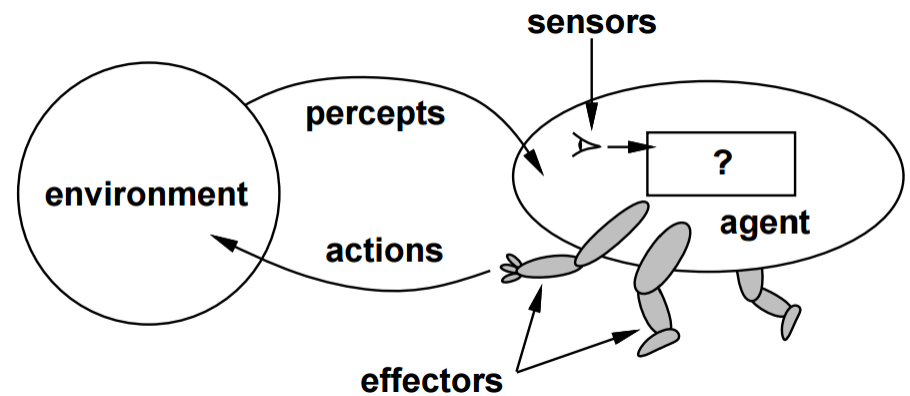
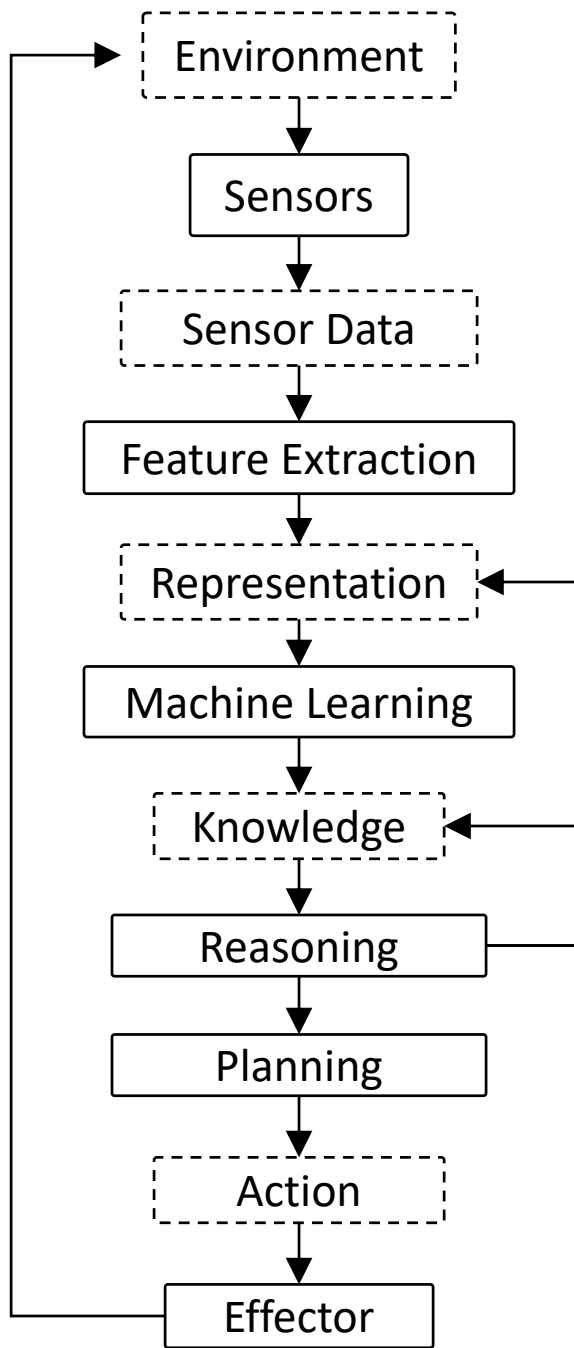
Success

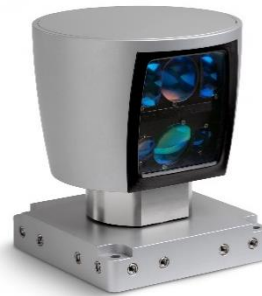
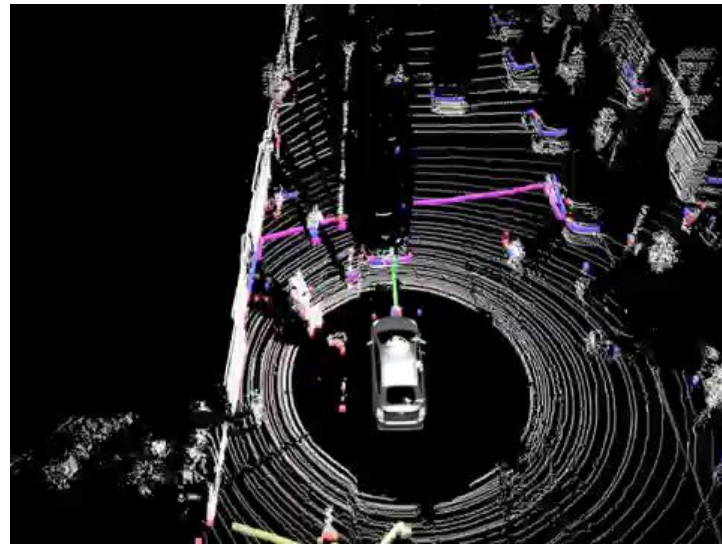
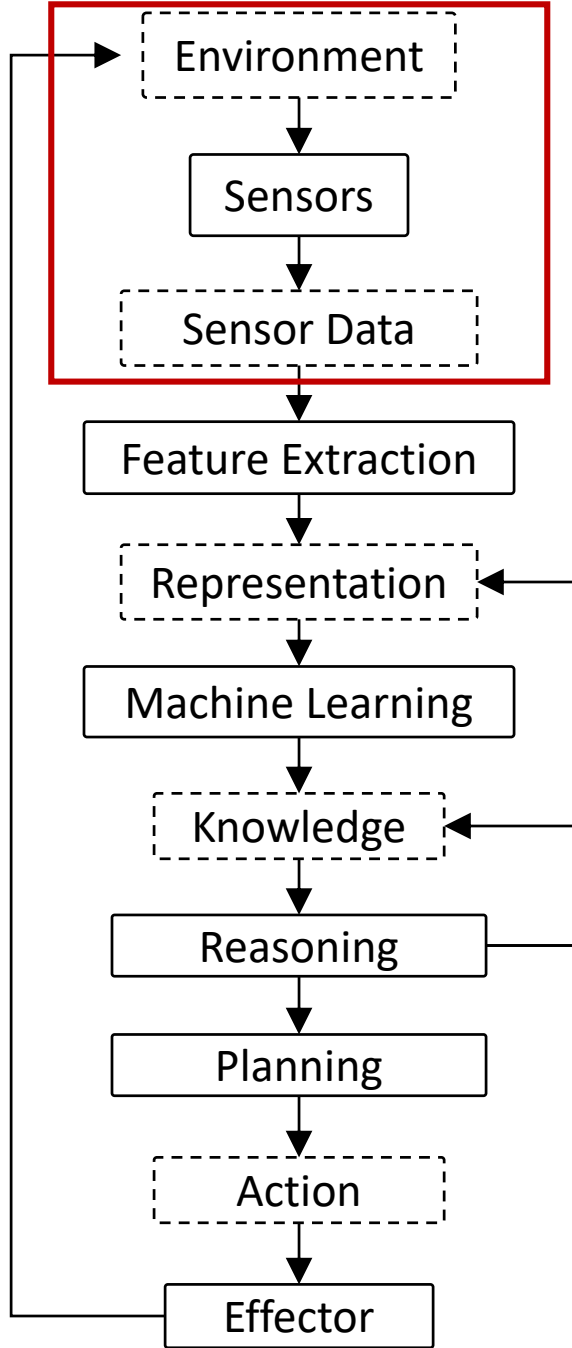
Reinforcement Learning in Humans

- Human appear to learn to walk through “very few examples” of trial and error. **How** is an open question...
- Possible answers:
 - **Hardware:** 230 million years of bipedal movement data.
 - **Imitation Learning:** Observation of other humans walking.
 - **Algorithms:** Better than backpropagation and stochastic gradient descent



Open Question: What can be learned from data?





Lidar



Camera
(Visible, Infrared)



Radar



GPS



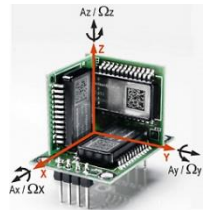
Stereo Camera



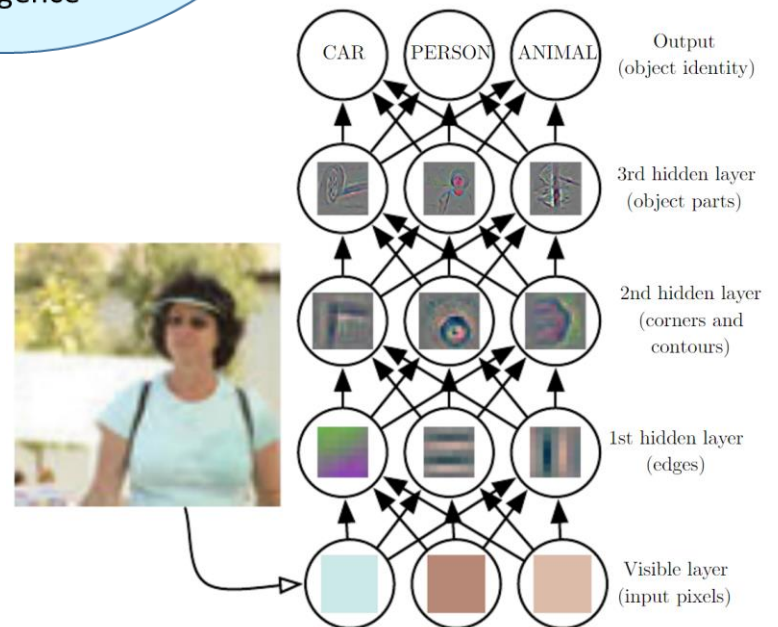
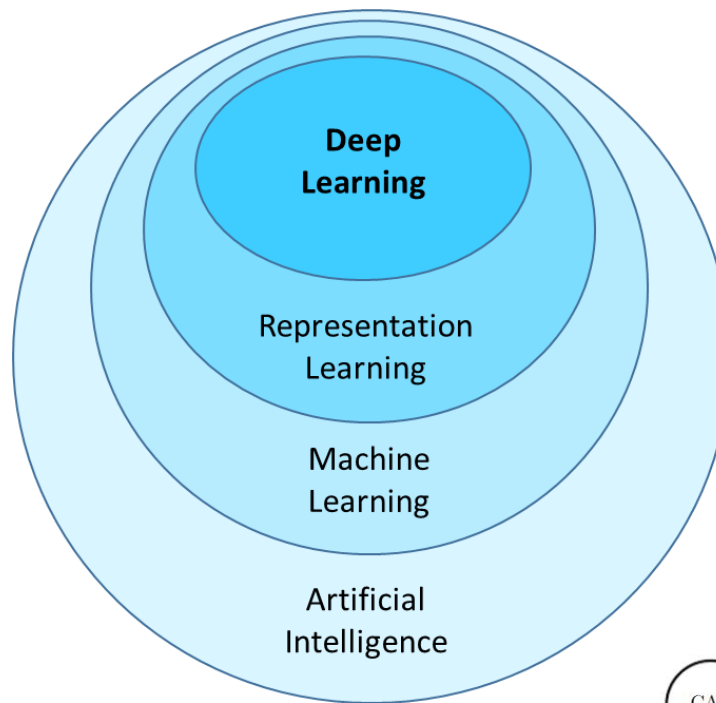
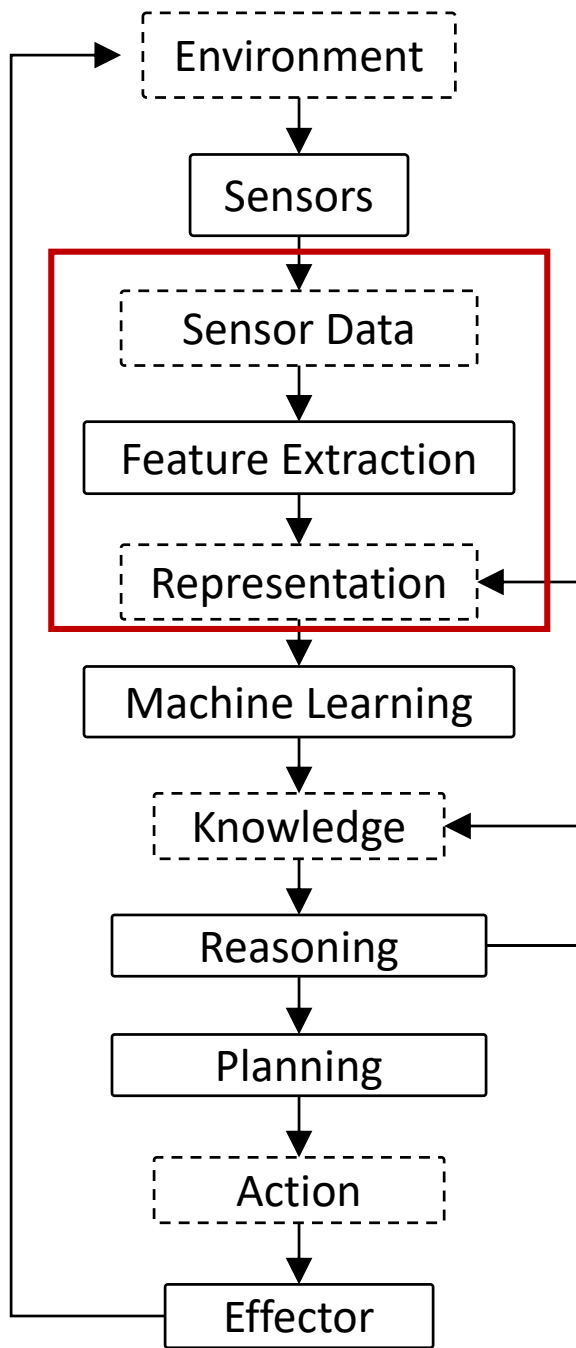
Microphone

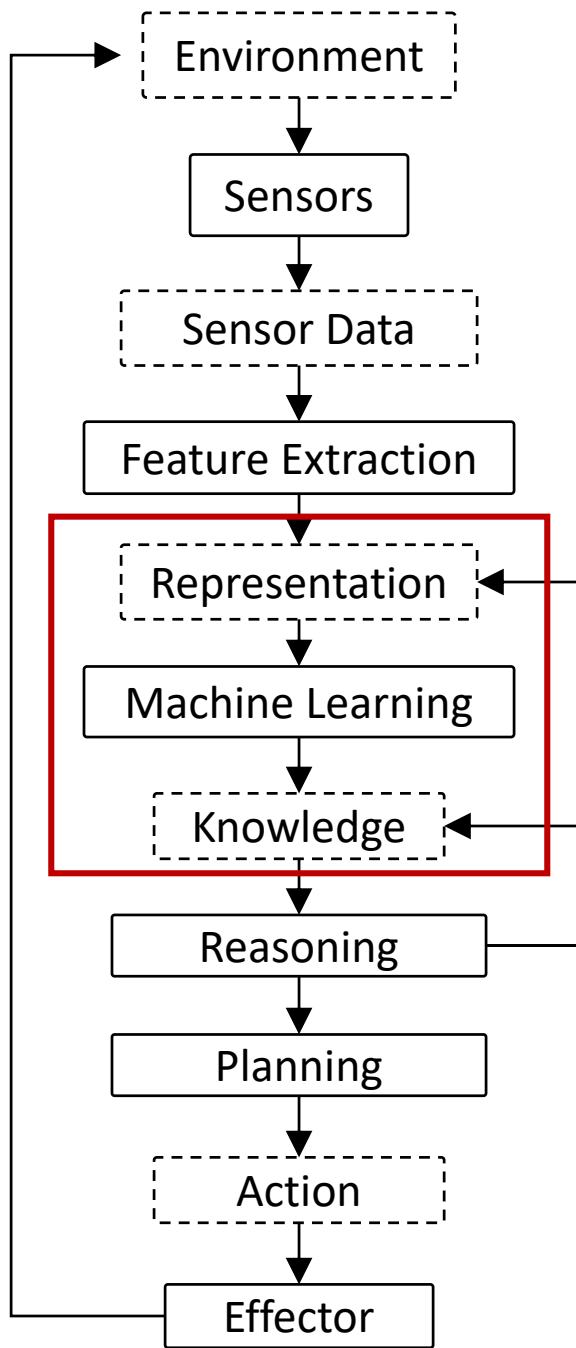


Networking
(Wired, Wireless)

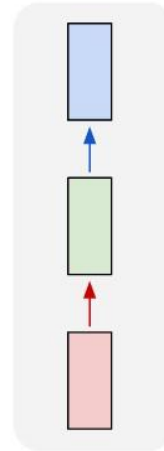


IMU

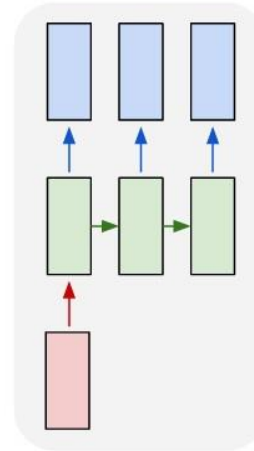




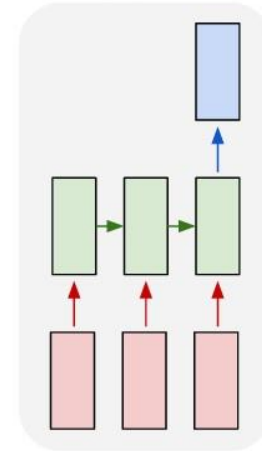
one to one



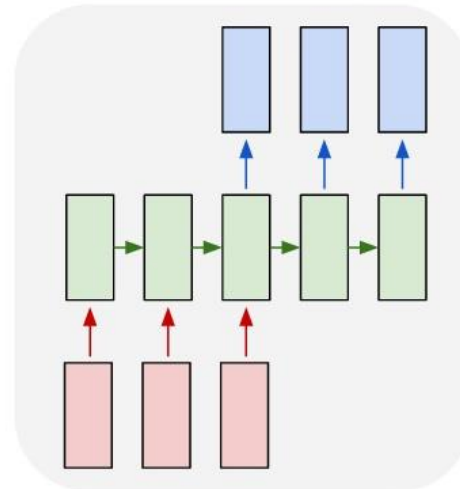
one to many



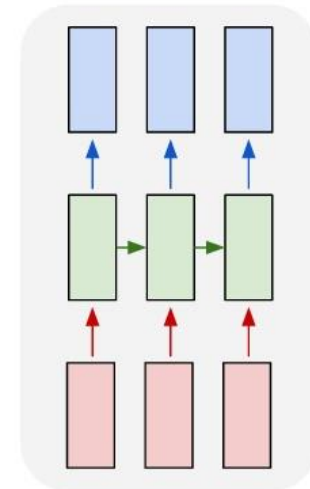
many to one



many to many



many to many



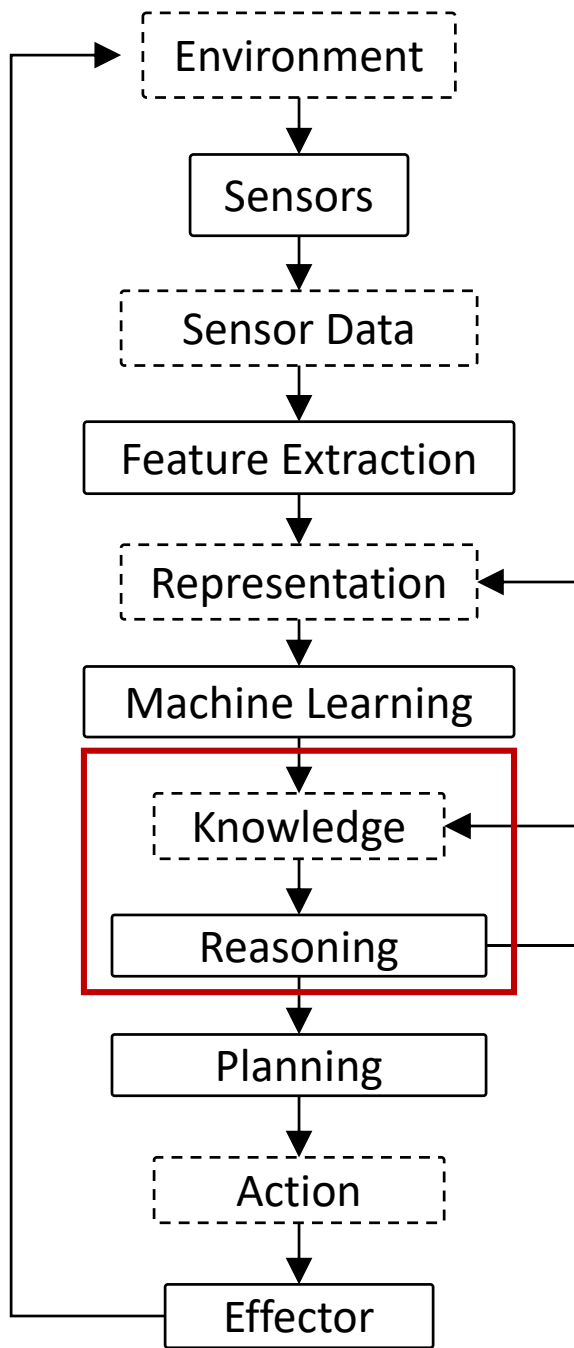
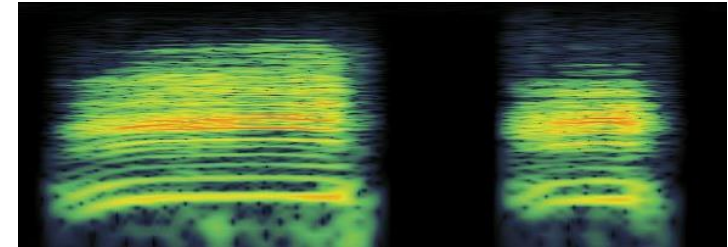


Image Recognition:
If it looks like a duck

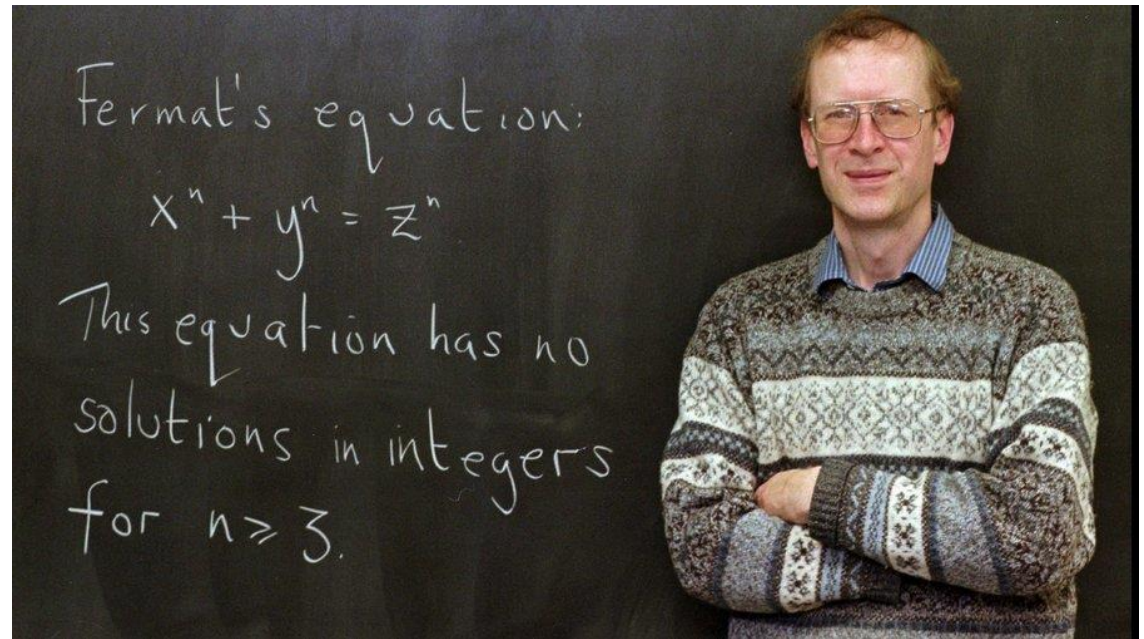
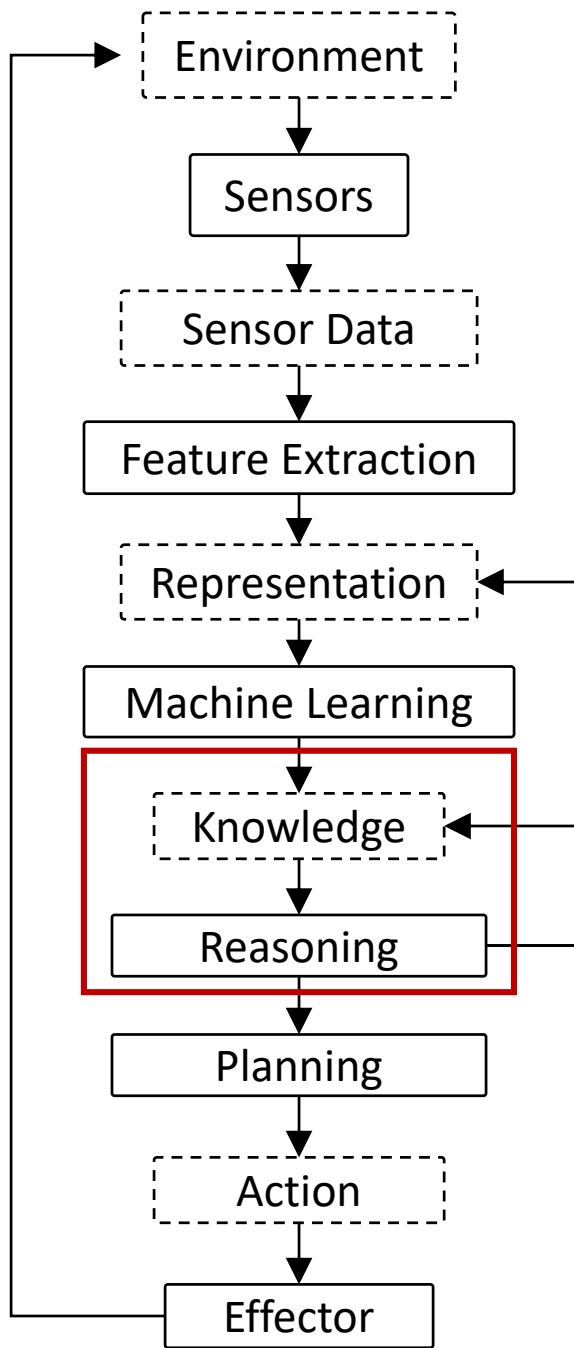


Audio Recognition:
Quacks like a duck

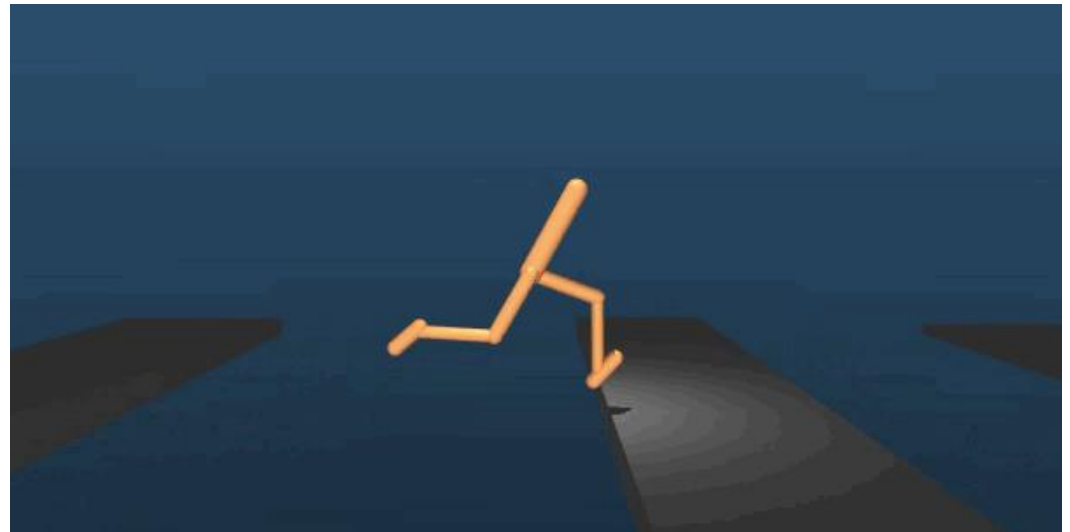
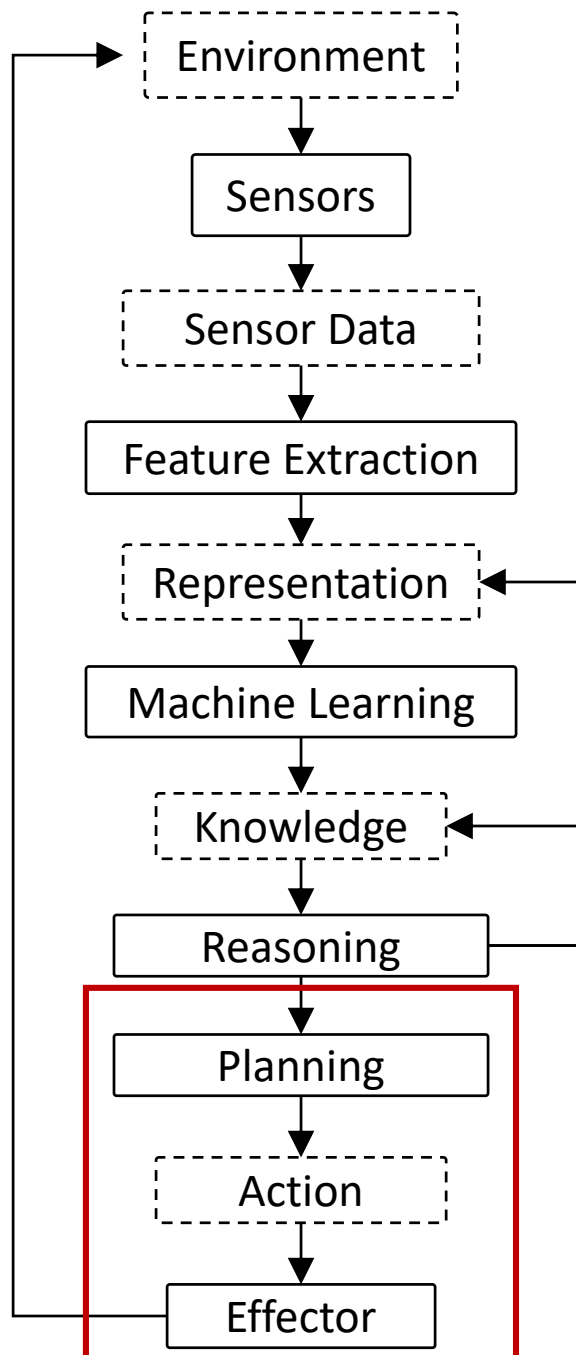


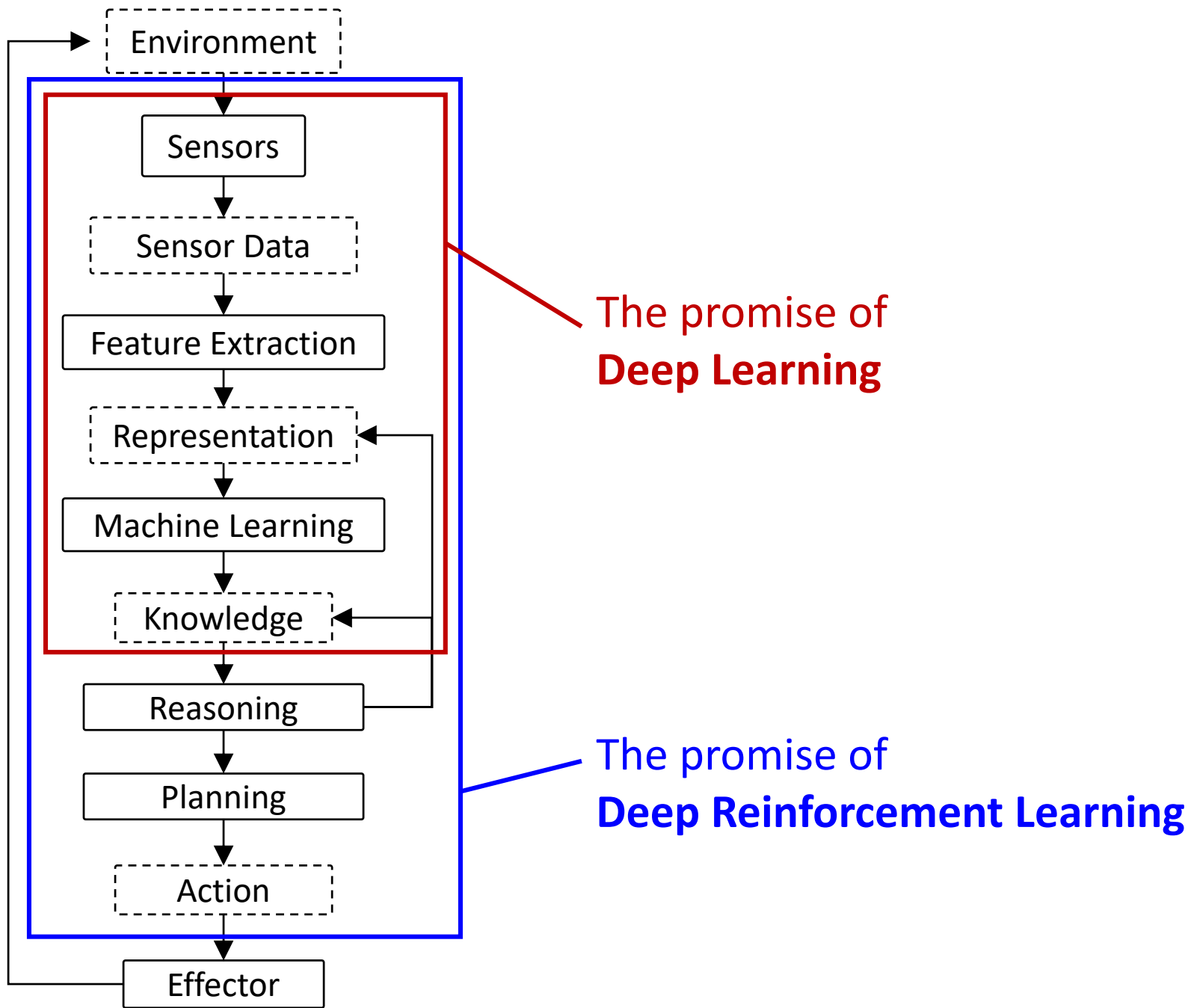
Activity Recognition:
Swims like a duck





Final **breakthrough**, 358 years after its conjecture:
“It was so indescribably beautiful; it was so simple and so elegant. I couldn’t understand how I’d missed it and I just stared at it in disbelief for twenty minutes. Then during the day I walked around the department, and I’d keep coming back to my desk looking to see if it was still there. It was still there. I couldn’t contain myself, I was so excited. It was the most important moment of my working life. Nothing I ever do again will mean as much.”





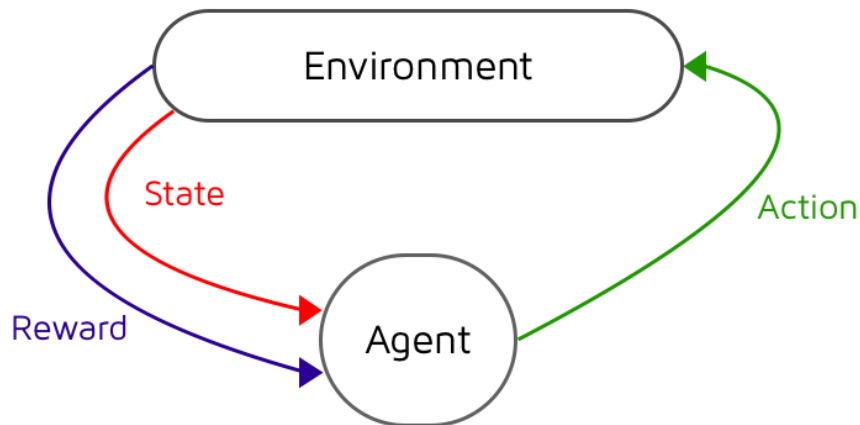
Reinforcement Learning Framework

At each step, the agent:

- Executes **action**
- Observe new **state**
- Receive **reward**

Open Questions:

- What cannot be modeled in this way?
- What are the challenges of learning in this framework?



Environment and Actions

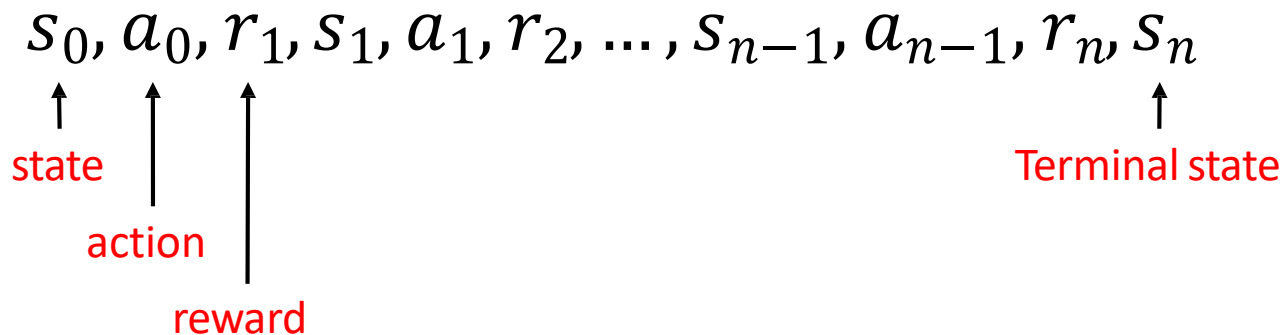
- Fully Observable (Chess) vs Partially Observable (Poker)
- Single Agent (Atari) vs Multi Agent (DeepTraffic)
- Deterministic (Cart Pole) vs Stochastic (DeepTraffic)
- Static (Chess) vs Dynamic (DeepTraffic)
- Discrete (Chess) vs Continuous (Cart Pole)

Note: *Real-world environment might not technically be stochastic or partially-observable but might as well be treated as such due to their complexity.*

Major Components of an RL Agent

An RL agent may be directly or indirectly trying to learn a:

- **Policy:** agent's behavior function
- **Value function:** how good is each state and/or action
- **Model:** agent's representation of the environment



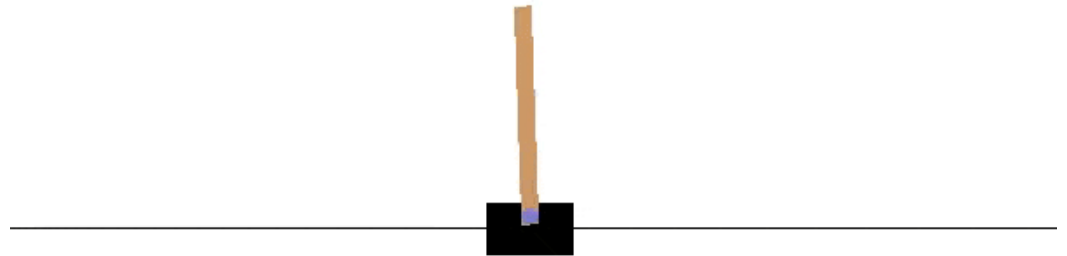
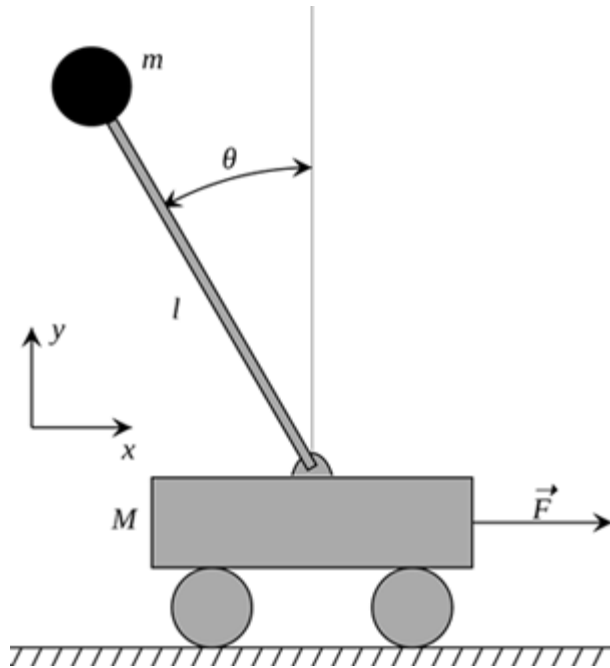
Meaning of Life for RL Agent: **Maximize Reward**

- Future reward: $R_t = r_t + r_{t+1} + r_{t+2} + \dots + r_n$
- Discounted future reward:

$$R_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots + \gamma^{n-t} r_n$$

- A good strategy for an agent would be to always choose an action that **maximizes the (discounted) future reward**
- Why “discounted”?
 - Math trick to help analyze convergence
 - Uncertainty due to environment stochasticity, partial observability, or that life can end at any moment:

Examples of Reinforcement Learning



Cart-Pole Balancing

- **Goal** — Balance the pole on top of a moving cart
- **State** — Pole angle, angular speed. Cart position, horizontal velocity.
- **Actions** — horizontal force to the cart
- **Reward** — 1 at each time step if the pole is upright

Examples of Reinforcement Learning

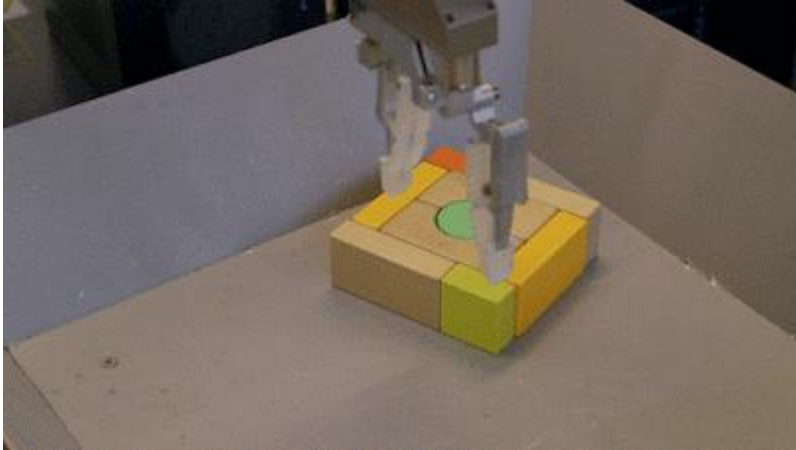
Doom*

- **Goal:**
Eliminate all opponents
- **State:**
Raw game pixels of the game
- **Actions:**
Up, Down, Left, Right, Shoot, etc.
- **Reward:**
- Positive when eliminating an opponent,
negative when the agent is eliminated



** Added for important thought-provoking considerations of AI safety in the context of autonomous weapons systems (see AGI lectures on the topic).*

Examples of Reinforcement Learning



Grasping Objects with Robotic Arm

- **Goal** - Pick an object of different shapes
- **State** - Raw pixels from camera
- **Actions** – Move arm. Grasp.
- **Reward** - Positive when pickup is successful



Key Takeaways for Real-World Impact

- Deep Learning:
 - **Fun part:** Good algorithms that learn from data.
 - **Hard part:** Good questions, huge amounts of representative data.
- Deep Reinforcement Learning:
 - **Fun part:** Good algorithms that learn from data.
 - **Hard part:** Defining a useful state space, action space, and reward.
 - **Hardest part:** Getting meaningful data for the above formalization.

3 Types of Reinforcement Learning



Model-based

- Learn the model of the world, then plan using the model
- Update model often
- Re-plan often

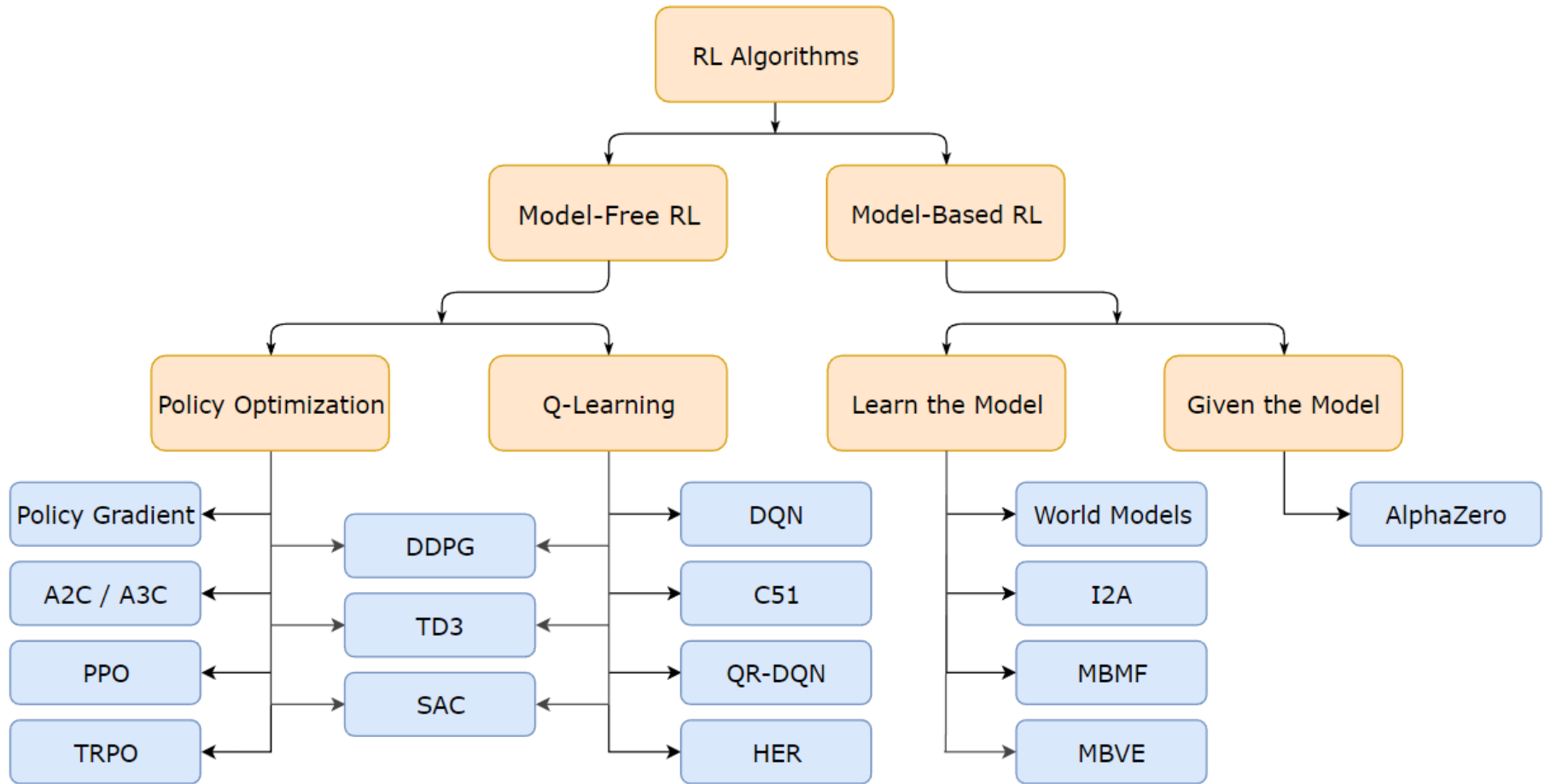
Value-based

- Learn the state or state-action value
- Act by choosing best action in state
- Exploration is a necessary add-on

Policy-based

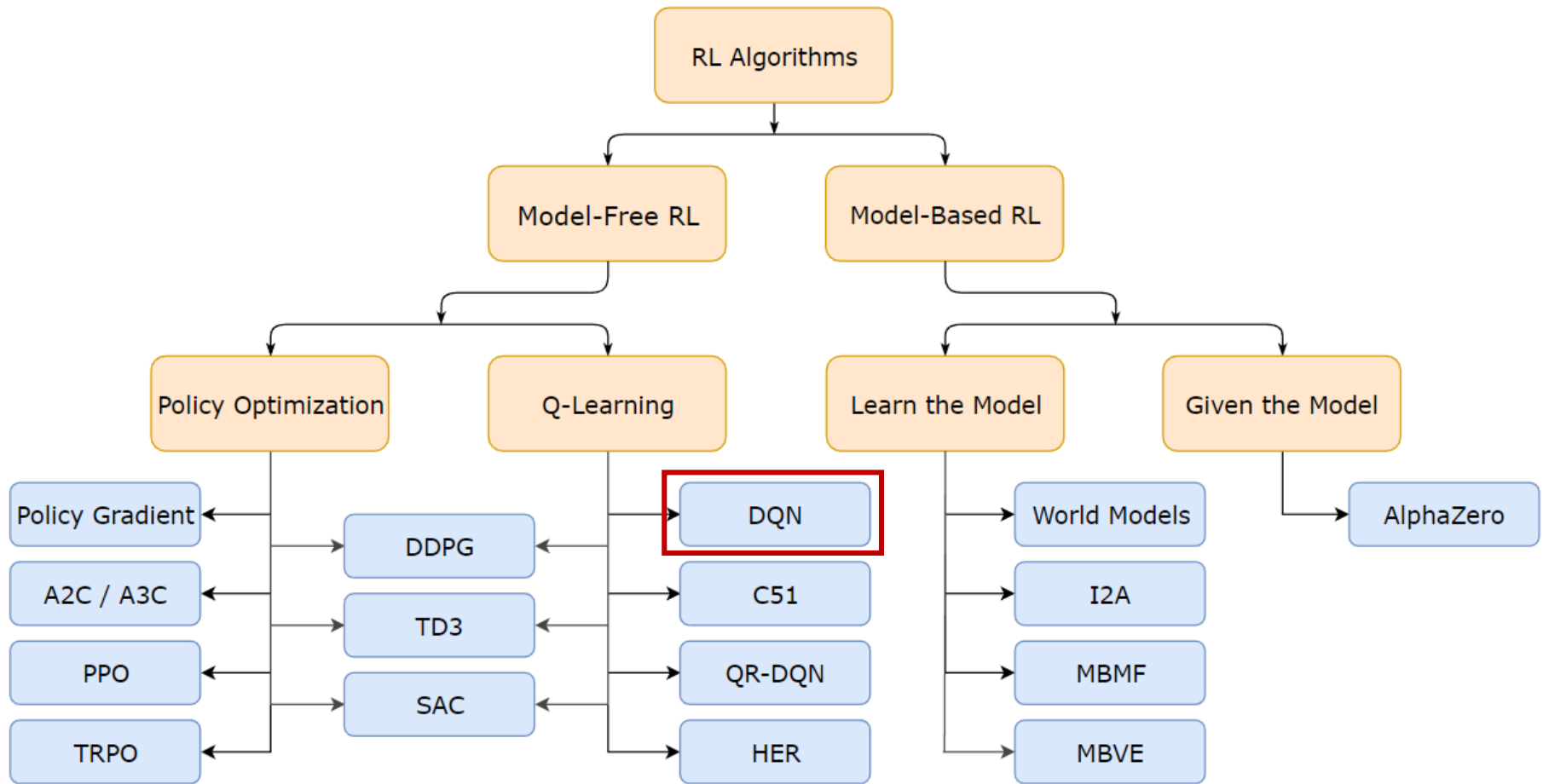
- Learn the stochastic policy function that maps state to action
- Act by sampling policy
- Exploration is baked in

Taxonomy of RL Methods



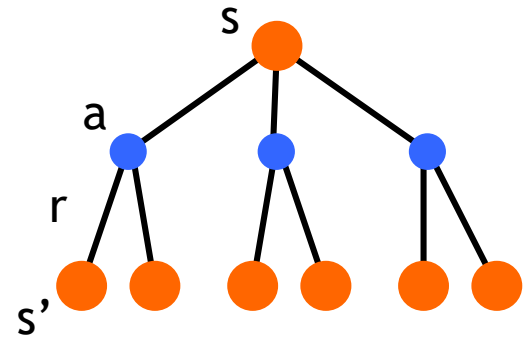
Link: <https://spinningup.openai.com>

Taxonomy of RL Methods



Q-Learning

- State-action value function: $Q^\pi(s,a)$
 - Expected return when starting in s , performing a , and following π



- Q-Learning: Use **any policy** to estimate Q that maximizes future reward:
 - Q directly approximates Q^* (Bellman optimality equation)
 - Independent of the policy being followed
 - Only requirement: keep updating each (s,a) pair

$$Q_{t+1}(s_t, a_t) = Q_t(s_t, a_t) + \alpha \left(R_{t+1} + \gamma \max_a Q_t(s_{t+1}, a) - Q_t(s_t, a_t) \right)$$

Diagram illustrating the Q-Learning update equation with annotations:

- Learning Rate** (α) points to the learning rate parameter in the equation.
- Discount Factor** (γ) points to the discount factor parameter in the equation.
- New State** (s_t) points to the state s_t in the equation.
- Old State** (s_t) points to the state s_t in the equation.
- Reward** (R_{t+1}) points to the reward R_{t+1} in the equation.

Q-Learning: Value Iteration

Learning Rate Discount Factor

$$Q_{t+1}(s_t, a_t) = Q_t(s_t, a_t) + \alpha \left(R_{t+1} + \gamma \max_a Q_t(s_{t+1}, a) - Q_t(s_t, a_t) \right)$$

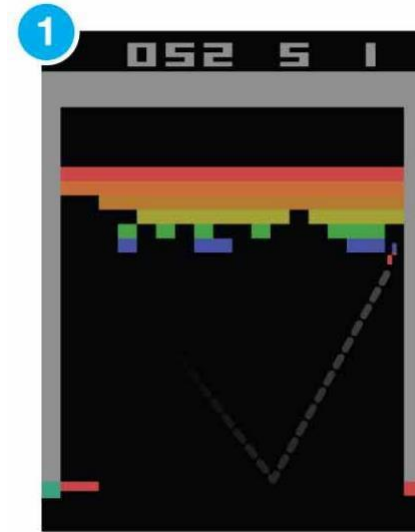
New State Old State Reward

	A1	A2	A3	A4
S1	+1	+2	-1	0
S2	+2	0	+1	-2
S3	-1	+1	0	-2
S4	-2	0	+1	+1

```
initialize  $Q[num\_states, num\_actions]$  arbitrarily
observe initial state  $s$ 
repeat
    select and carry out an action  $a$ 
    observe reward  $r$  and new state  $s'$ 
     $Q[s, a] = Q[s, a] + \alpha(r + \gamma \max_{a'} Q[s', a'] - Q[s, a])$ 
     $s = s'$ 
until terminated
```

Q-Learning: Representation Matters

- In practice, Value Iteration is impractical
 - Very limited states/actions
 - Cannot generalize to unobserved states



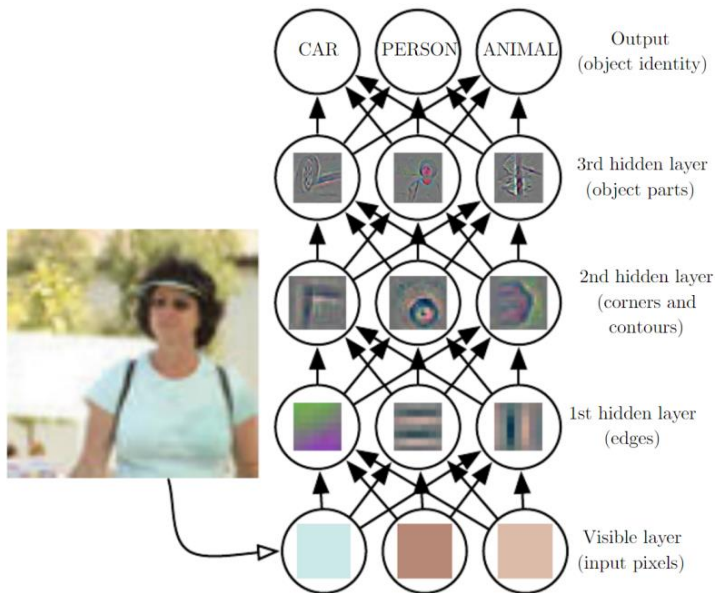
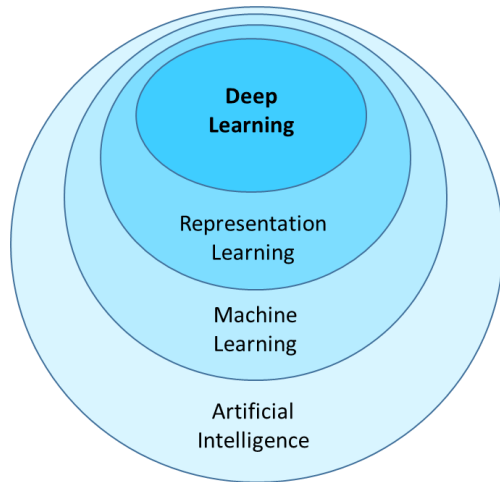
- Think about the **Breakout** game

- State: screen pixels
 - Image size: **84 × 84** (resized)
 - Consecutive **4** images
 - Grayscale with **256** gray levels

$256^{84 \times 84 \times 4}$ rows in the Q-table!

$= 10^{69,970} \gg 10^{82}$ atoms in the universe

Deep RL = RL + Neural Networks



Representation Matters

Representation:
The Earth is fixed center of
our Solar System

Representation:
The Sun is fixed center of
our Solar System



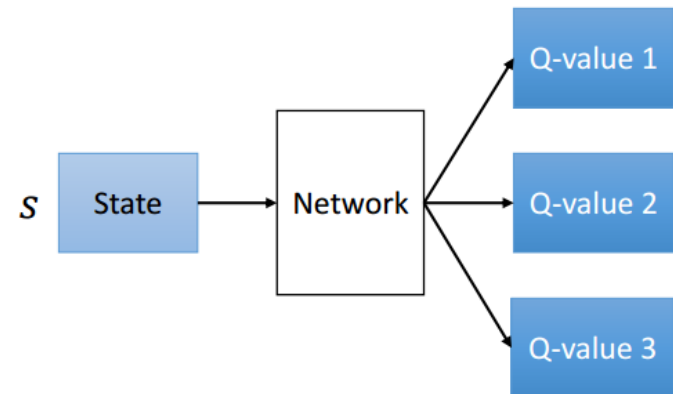
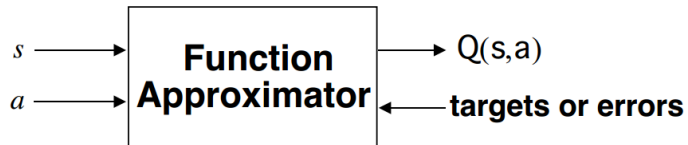
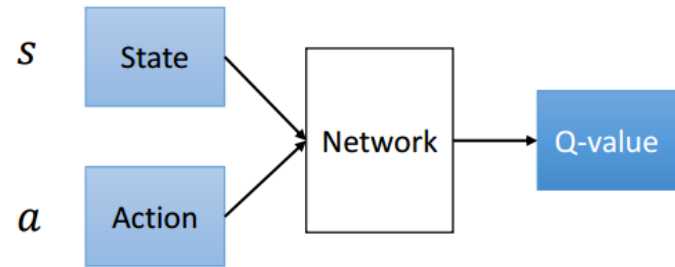
Geocentric Model
(Anaximander, 6th century BC)

Heliocentric Model
(Copernicus, 1543)

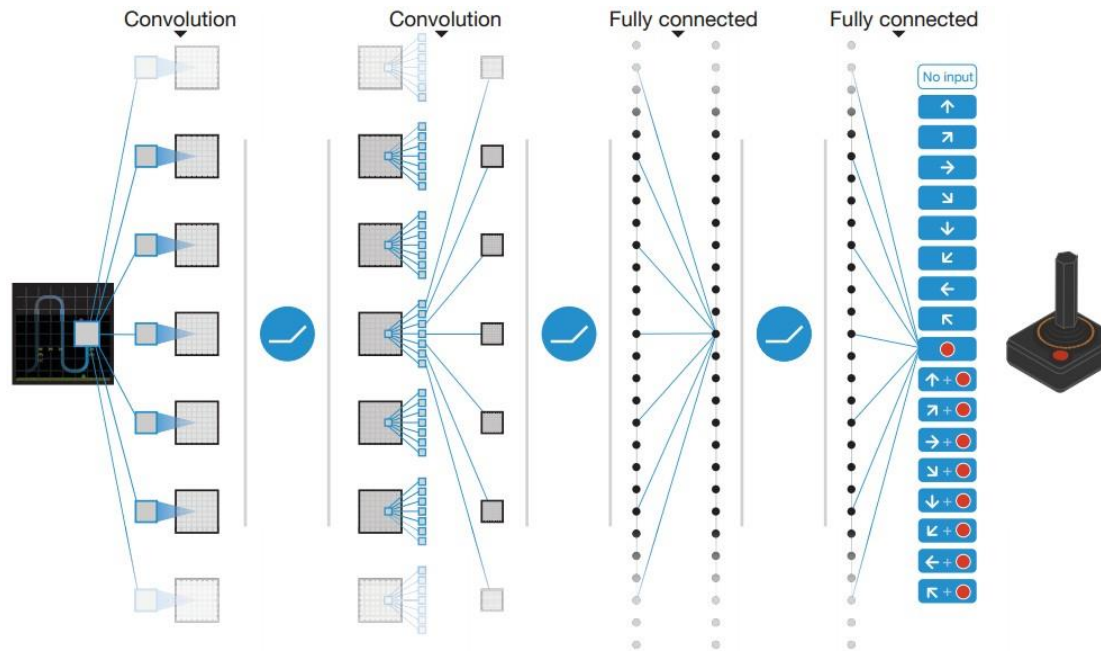
DQN: Deep Q-Learning

Use a neural network to approximate the Q-function:

$$Q(s, a; \theta) \approx Q^*(s, a)$$



Deep Q-Network (DQN): Atari



Layer	Input	Filter size	Stride	Num filters	Activation	Output
conv1	84x84x4	8x8	4	32	ReLU	20x20x32
conv2	20x20x32	4x4	2	64	ReLU	9x9x64
conv3	9x9x64	3x3	1	64	ReLU	7x7x64
fc4	7x7x64			512	ReLU	512
fc5	512			18	Linear	18

Mnih et al. "Playing atari with deep reinforcement learning." 2013.

DQN and Double DQN

- Loss function (squared error):

$$L = \mathbb{E}[\underbrace{(r + \gamma \max_{a'} Q(s', a'))}_{\text{target}} - \underbrace{Q(s, a)}_{\text{prediction}})^2]$$

- DQN: same network for both Q
- Double DQN: separate network for each Q
 - Helps reduce bias introduced by the inaccuracies of Q network at the beginning of training

DQN Tricks

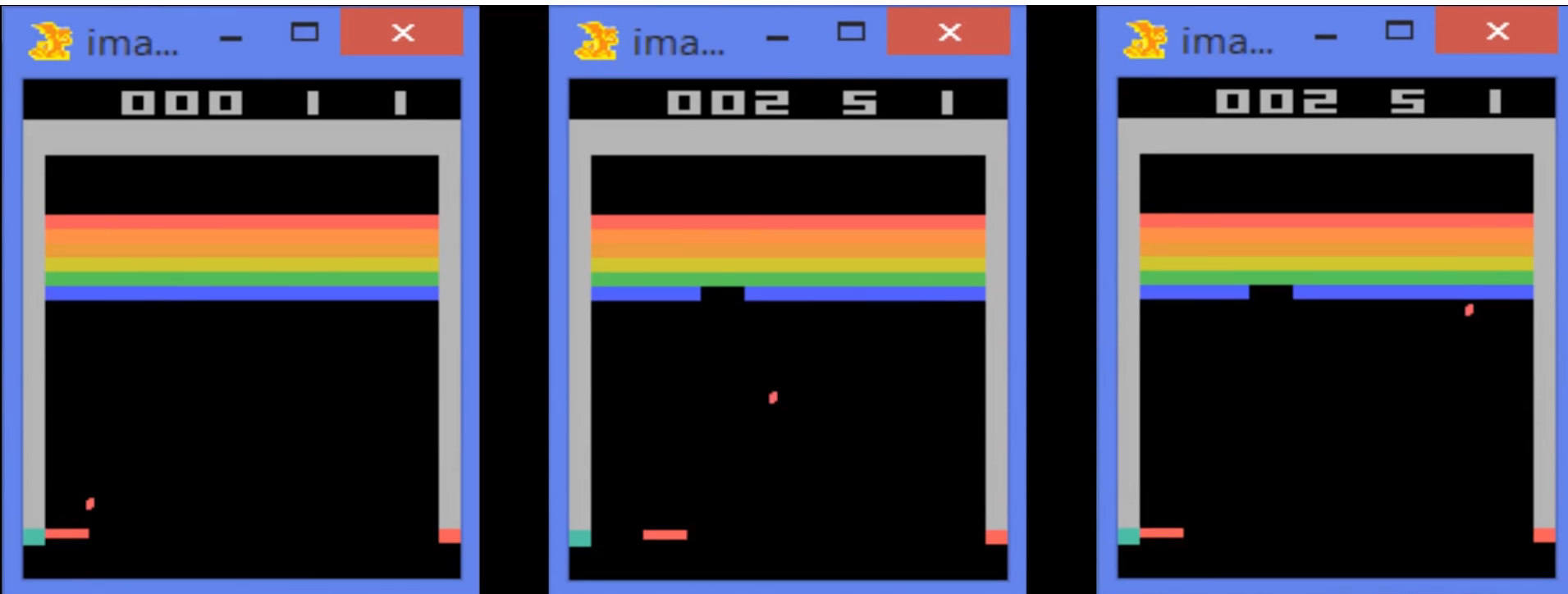
- Experience Replay
 - Stores experiences (actions, state transitions, and rewards) and creates mini-batches from them for the training process
- Fixed Target Network
 - Error calculation includes the target function depends on network parameters and thus changes quickly. Updating it only every 1,000 steps increases stability of training process.

$$Q(s_t, a) \leftarrow Q(s_t, a) + \alpha \left[r_{t+1} + \gamma \max_p Q(s_{t+1}, p) - Q(s_t, a) \right]$$

target Q function in the red rectangular is fixed

Replay	○	○	×	×
Target	○	×	○	×
Breakout	316.8	240.7	10.2	3.2
River Raid	7446.6	4102.8	2867.7	1453.0
Seaquest	2894.4	822.6	1003.0	275.8
Space Invaders	1088.9	826.3	373.2	302.0

Atari Breakout

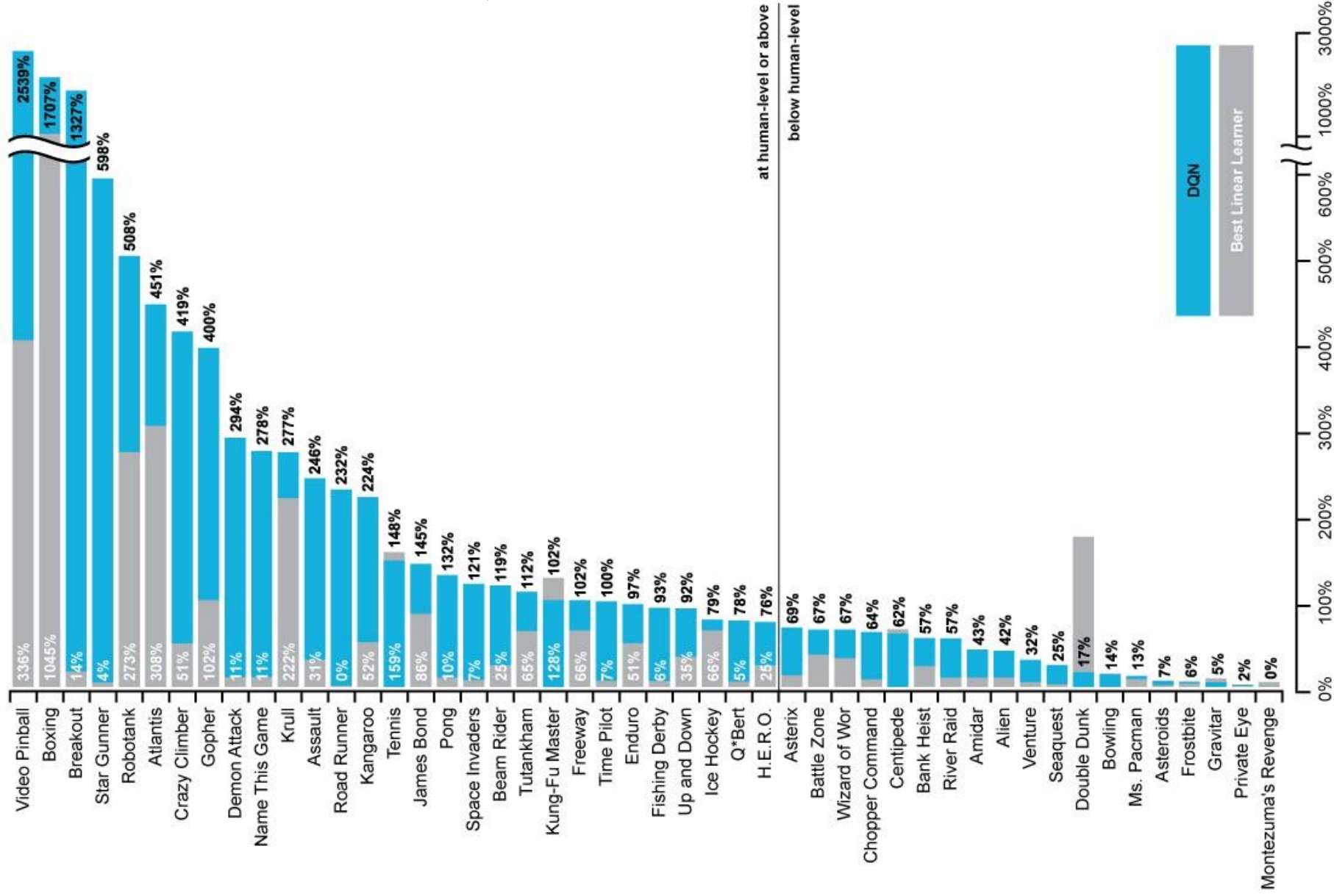


After
10 Minutes
of Training

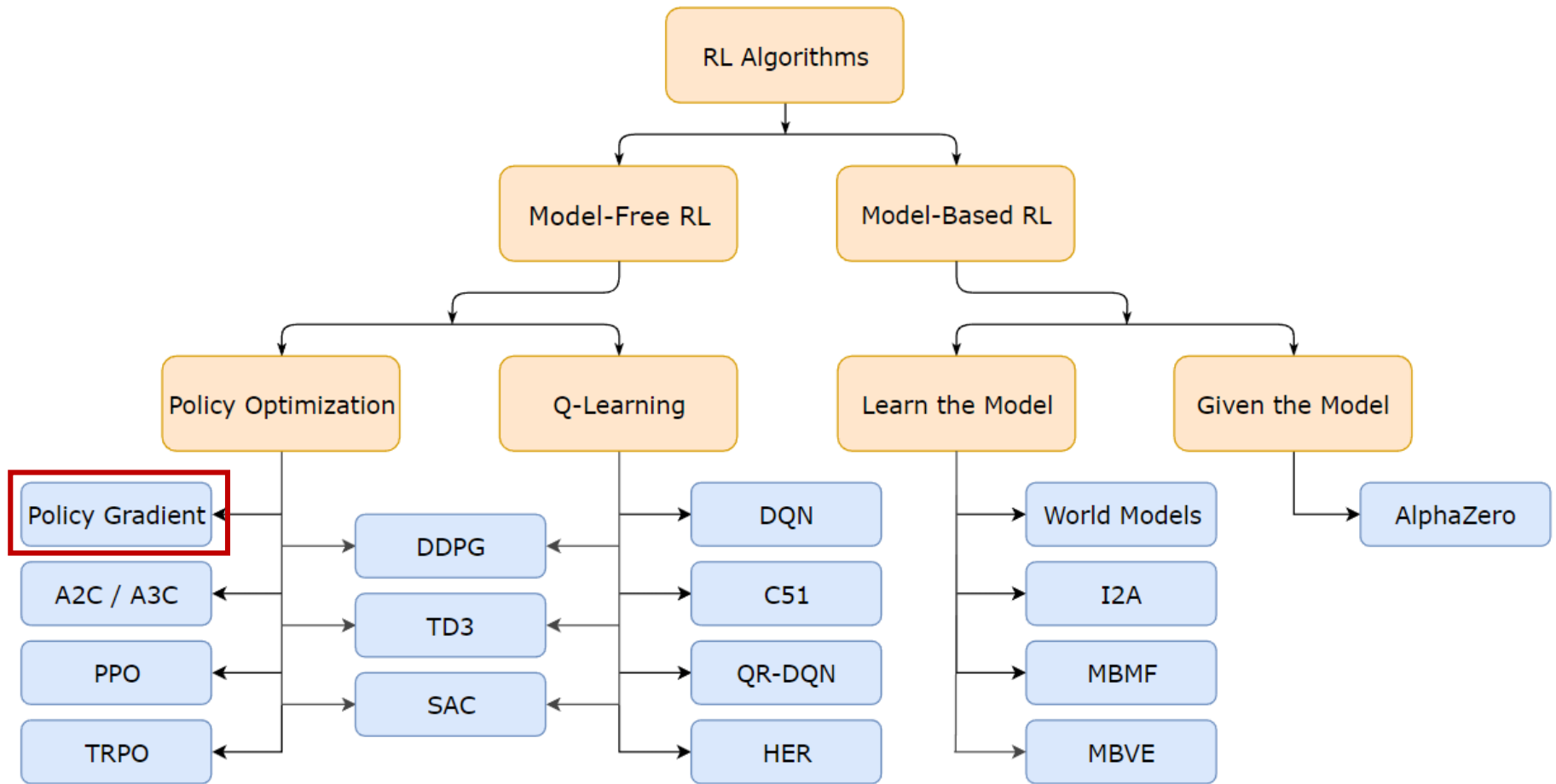
After
120 Minutes
of Training

After
240 Minutes
of Training

DQN Results in Atari

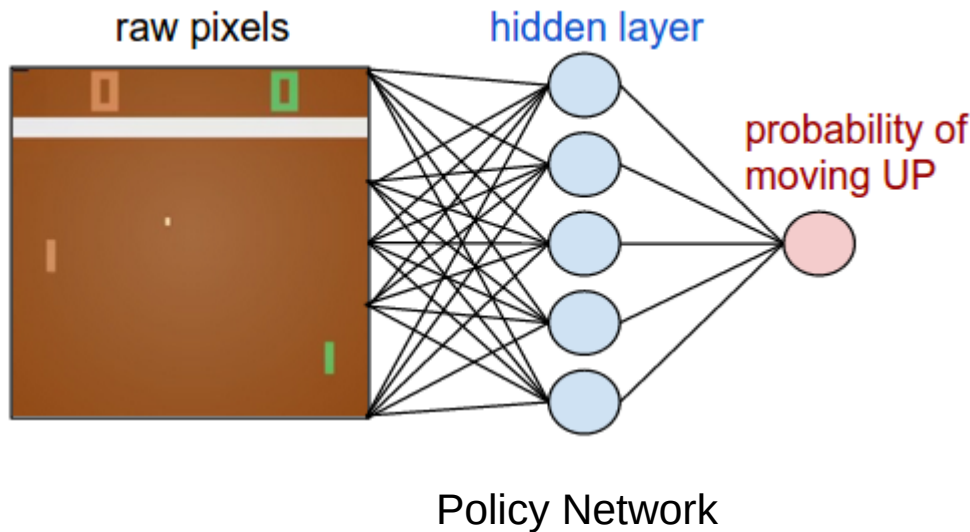


Taxonomy of RL Methods



Policy Gradient (PG)

- **DQN (off-policy):** Approximate Q and infer optimal policy
- **PG (on-policy):** Directly optimize policy space



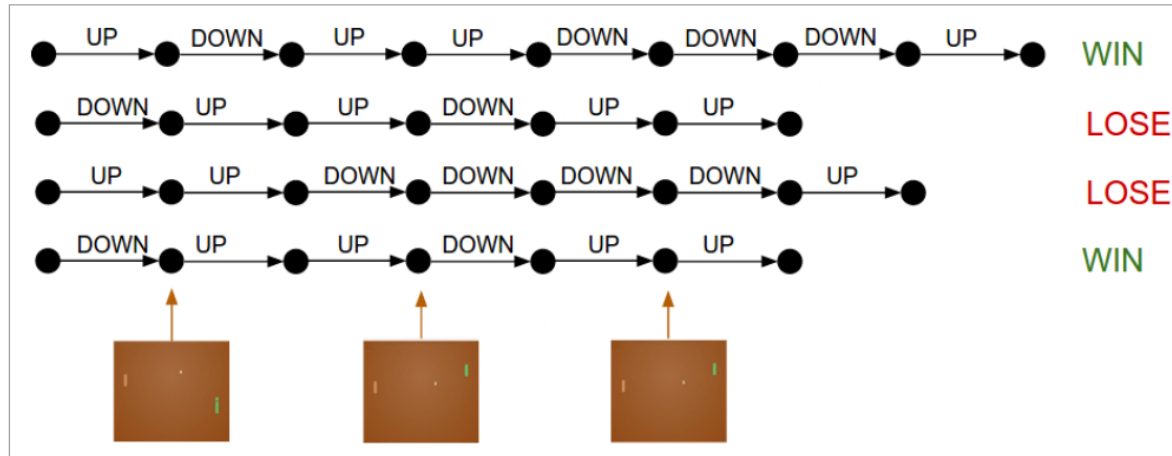
Good illustrative explanation:

<http://karpathy.github.io/2016/05/31/rl/>

*"Deep Reinforcement Learning:
Pong from Pixels"*

Policy Gradient – Training

Policy Gradients: Run a policy for a while. See what actions led to high rewards. Increase their probability.



- **REINFORCE:** Policy gradient that increases probability of good actions and decreases probability of bad action:

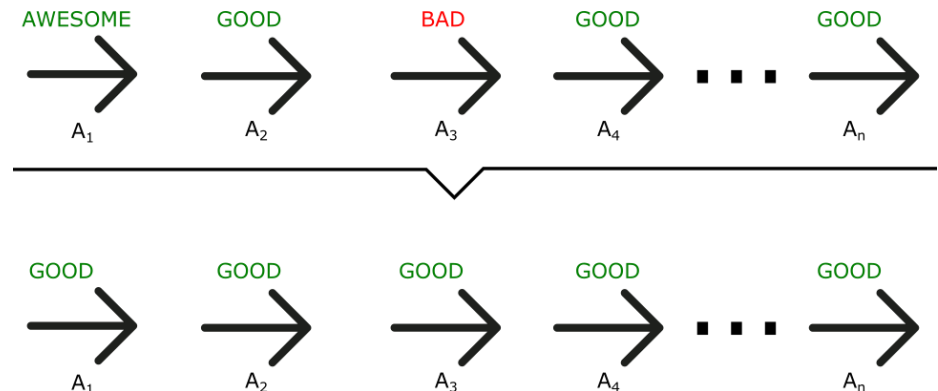
$$\nabla_{\theta} E[R_t] = E[\nabla_{\theta} \log P(a) R_t]$$

Policy Gradient

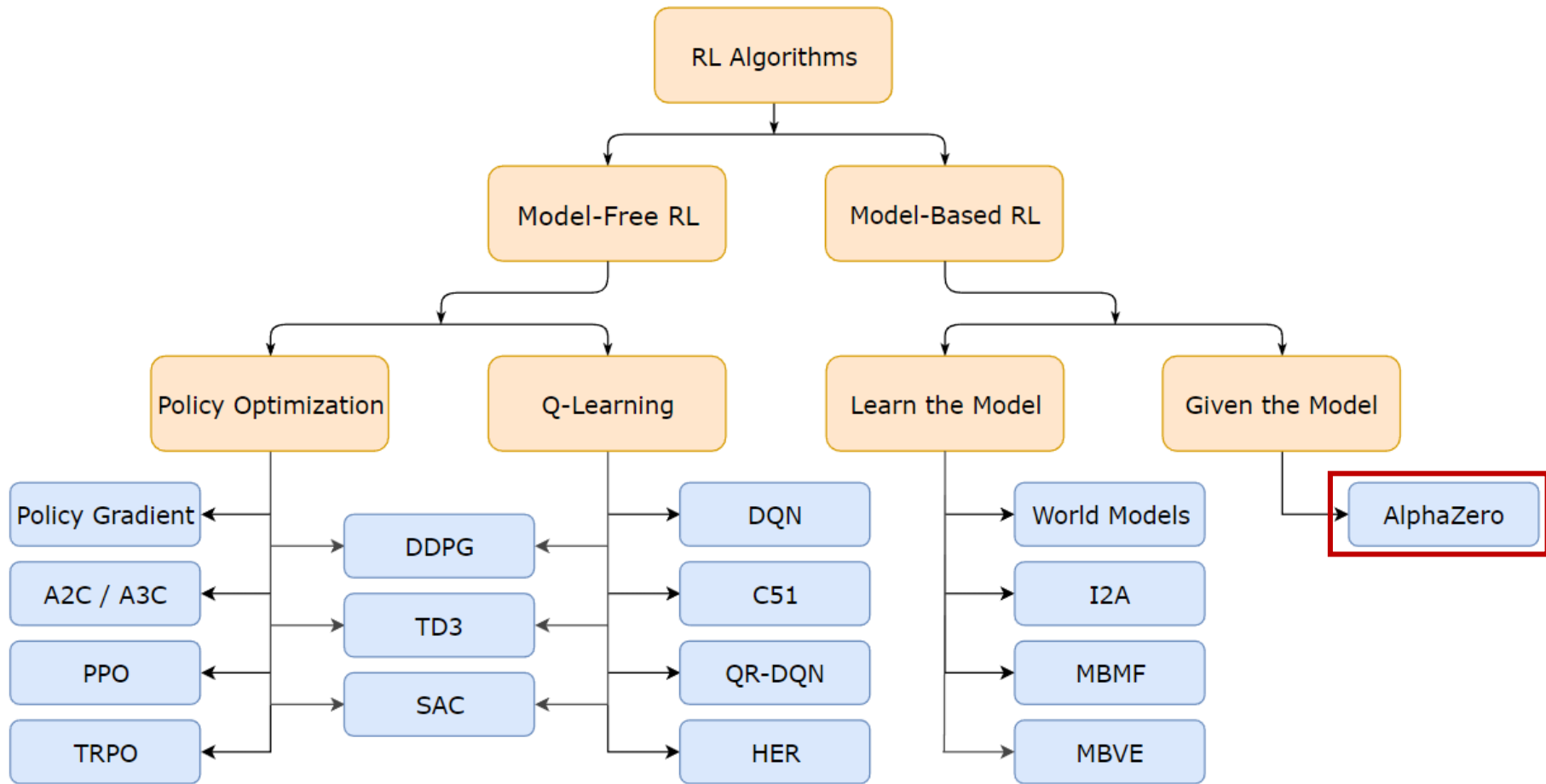
- Pros vs DQN:
 - **Messy World:** If Q function is too complex to be learned, DQN may fail miserably, while PG will still learn a good policy.
 - **Speed:** Faster convergence
 - **Stochastic Policies:** Capable of learning stochastic policies - DQN can't
 - **Continuous actions:** Much easier to model continuous action space
- Cons vs DQN:
 - **Data:** Sample inefficient (needs more data)
 - **Stability:** Less stable during training process.
 - Poor **credit assignment** to (state, action) pairs for delayed rewards

Problem with REINFORCE:

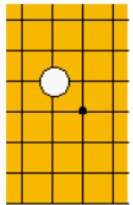
Calculating the reward at the end, means all the actions will be averaged as good because the total reward was high.



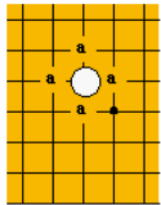
Taxonomy of RL Methods



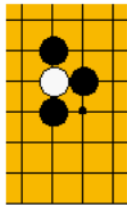
Game of Go



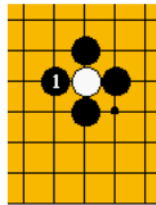
Dia. 3



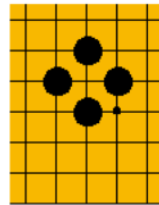
Dia. 4
liberties



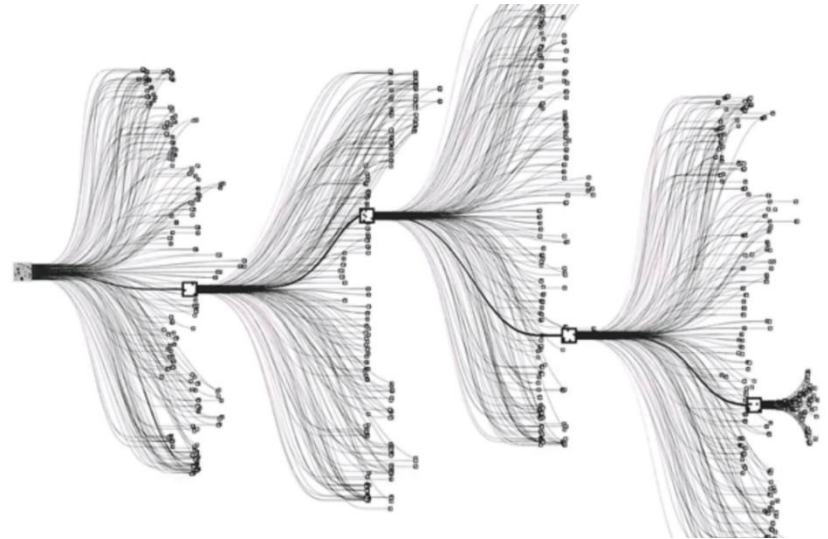
Dia. 5
atari



Dia. 6
capture

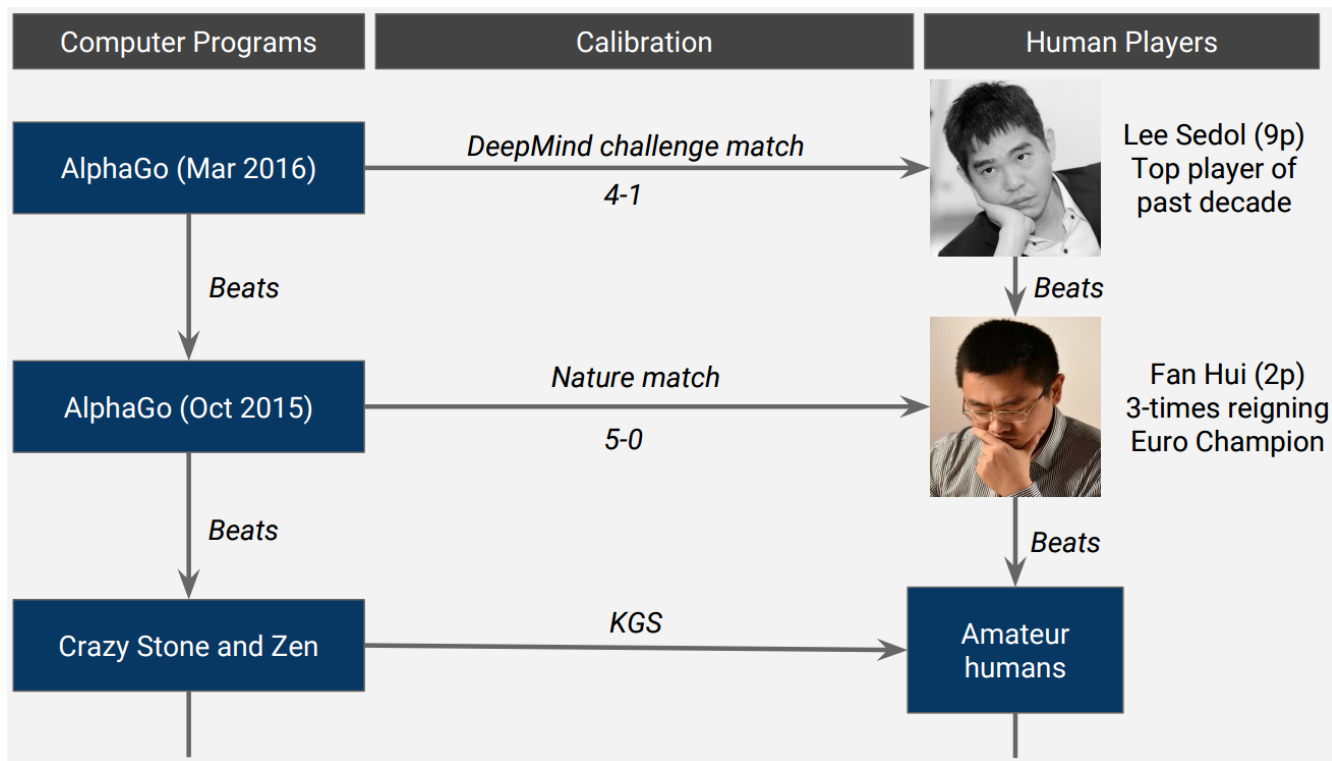
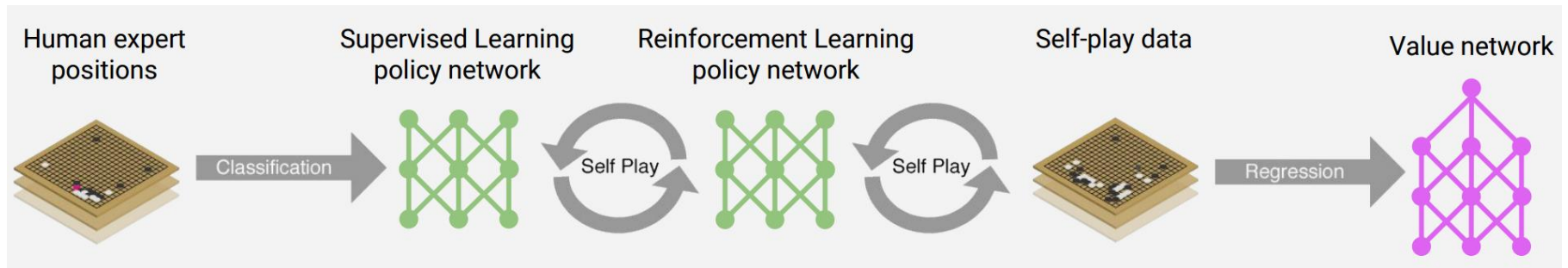


Dia. 7
result

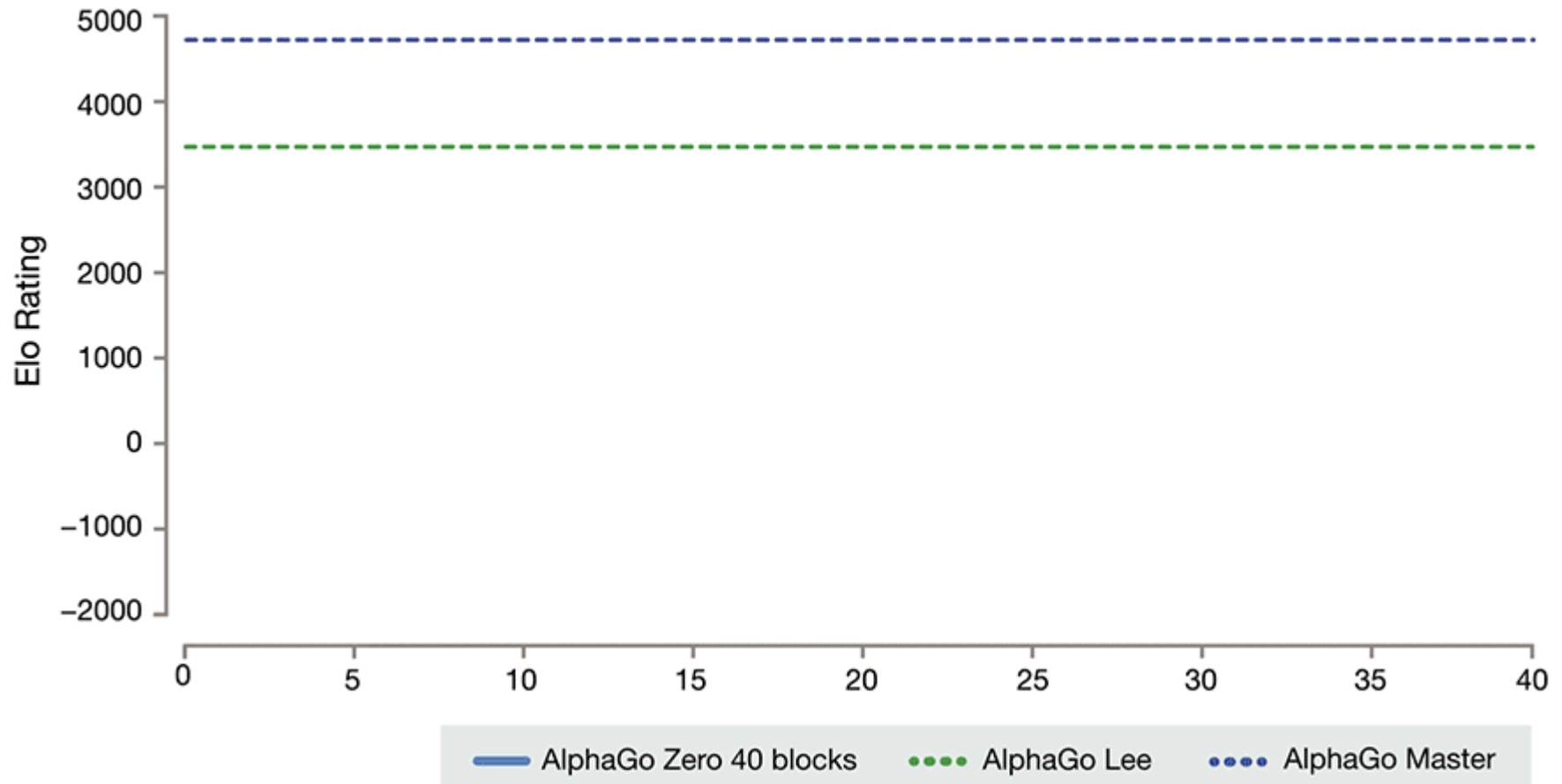


Game size	Board size N	3^N	Percent legal	legal game positions (A094777) ^[11]
1×1	1	3	33%	1
2×2	4	81	70%	57
3×3	9	19,683	64%	12,675
4×4	16	43,046,721	56%	24,318,165
5×5	25	8.47×10^{11}	49%	4.1×10^{11}
9×9	81	4.4×10^{38}	23.4%	1.039×10^{38}
13×13	169	4.3×10^{80}	8.66%	$3.72497923 \times 10^{79}$
19×19	361	1.74×10^{172}	1.196%	$2.08168199382 \times 10^{170}$

AlphaGo (2016) Beat Top Human at Go

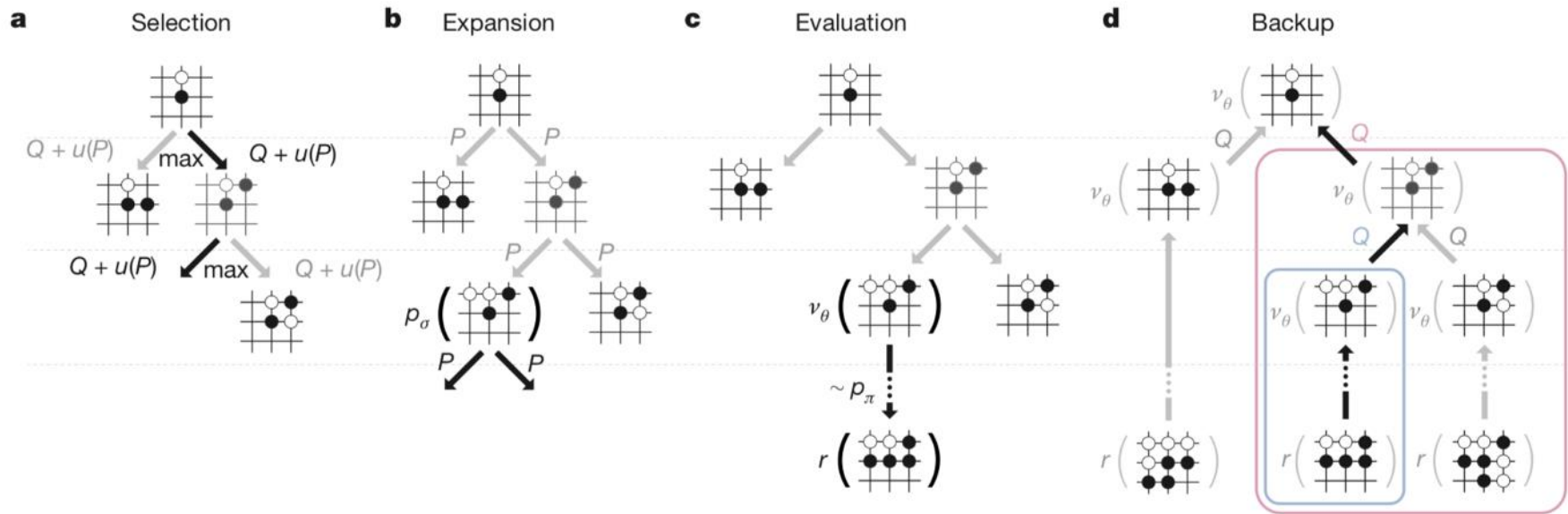


AlphaGo Zero (2017): Beats AlphaGo



AlphaGo Zero Approach

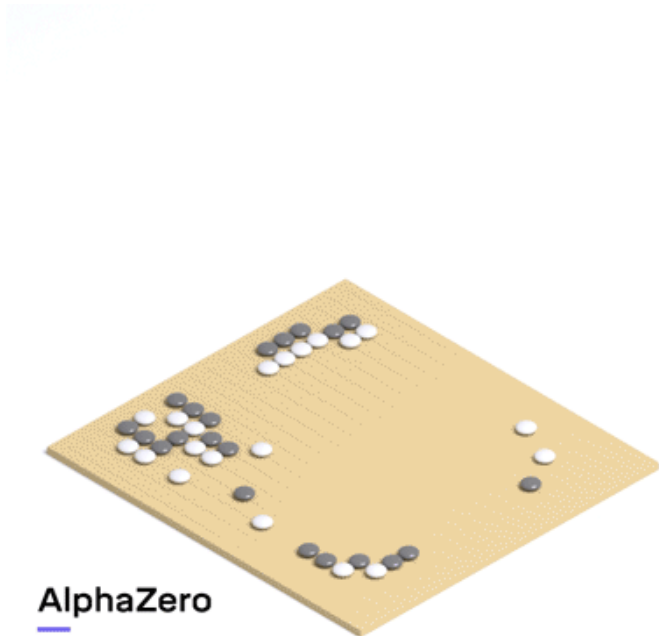
- Same as the best before: Monte Carlo Tree Search (MCTS)
 - Balance exploitation/exploration (going deep on promising positions or exploring new underplayed positions)
- Use a neural network as “intuition” for which positions to expand as part of MCTS (same as AlphaGo)



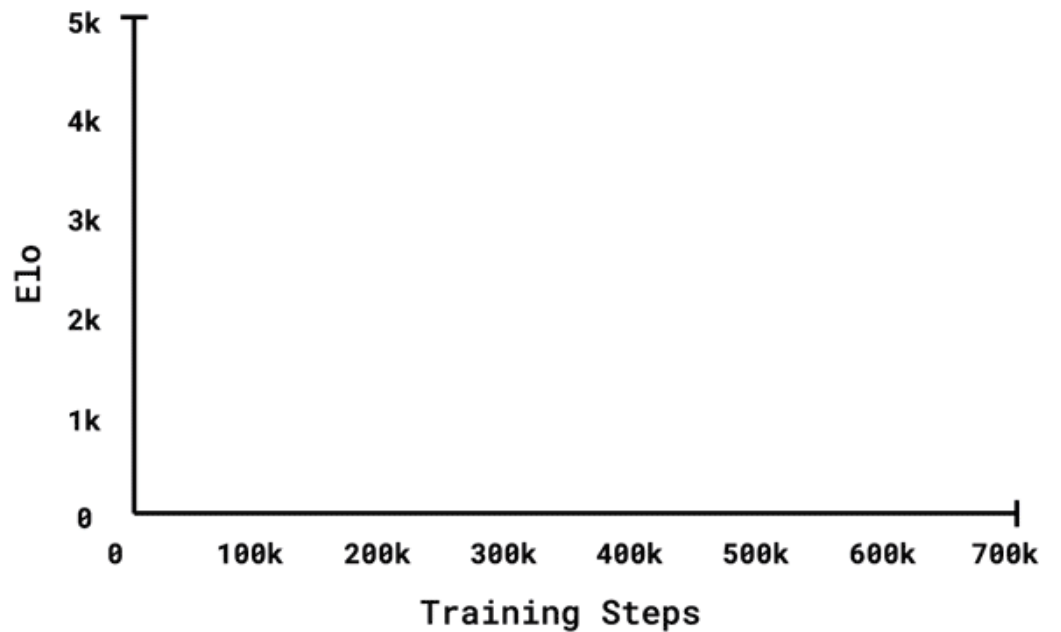
AlphaGo Zero Approach

- Same as the best before: Monte Carlo Tree Search (MCTS)
 - Balance exploitation/exploration (going deep on promising positions or exploring new underplayed positions)
- Use a neural network as “intuition” for which positions to expand as part of MCTS (same as AlphaGo)
- “Tricks”
 - Use MCTS intelligent look-ahead (instead of human games) to improve value estimates of play options
 - Multi-task learning: “two-headed” network that outputs (1) move probability and (2) probability of winning.
 - Updated architecture: use residual networks

AlphaZero vs Chess, Shogi, Go

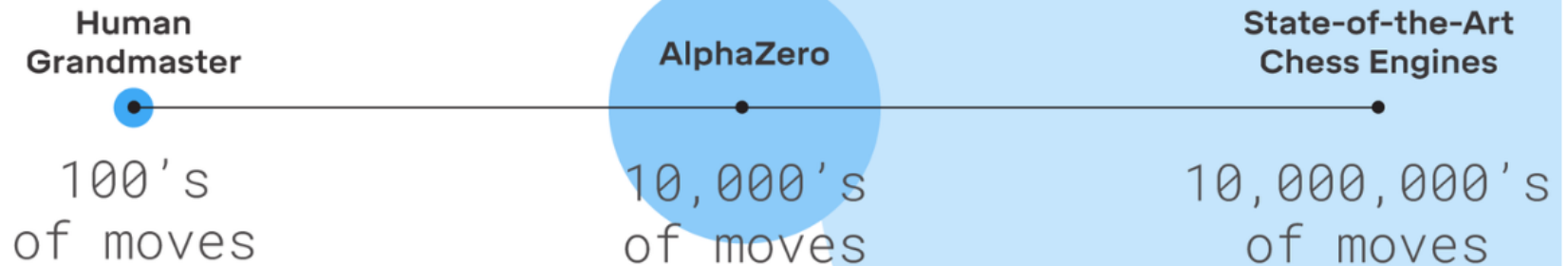


AlphaZero



AlphaZero vs Chess, Shogi, Go

Amount of Search per Decision

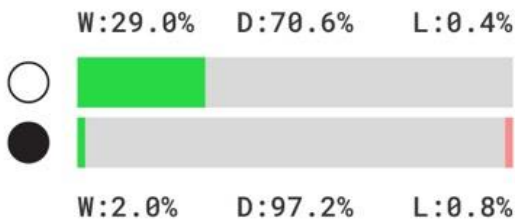


AlphaZero vs Chess, Shogi, Go

Chess



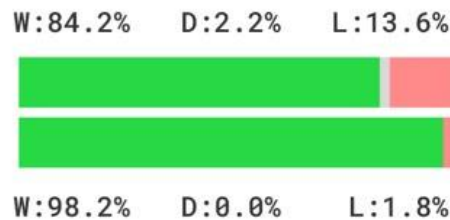
AlphaZero vs. Stockfish



Shogi



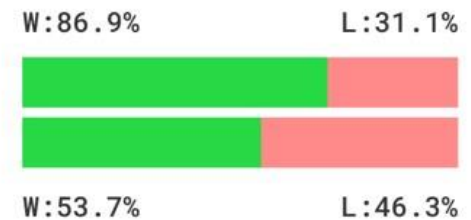
AlphaZero vs. Elmo



Go

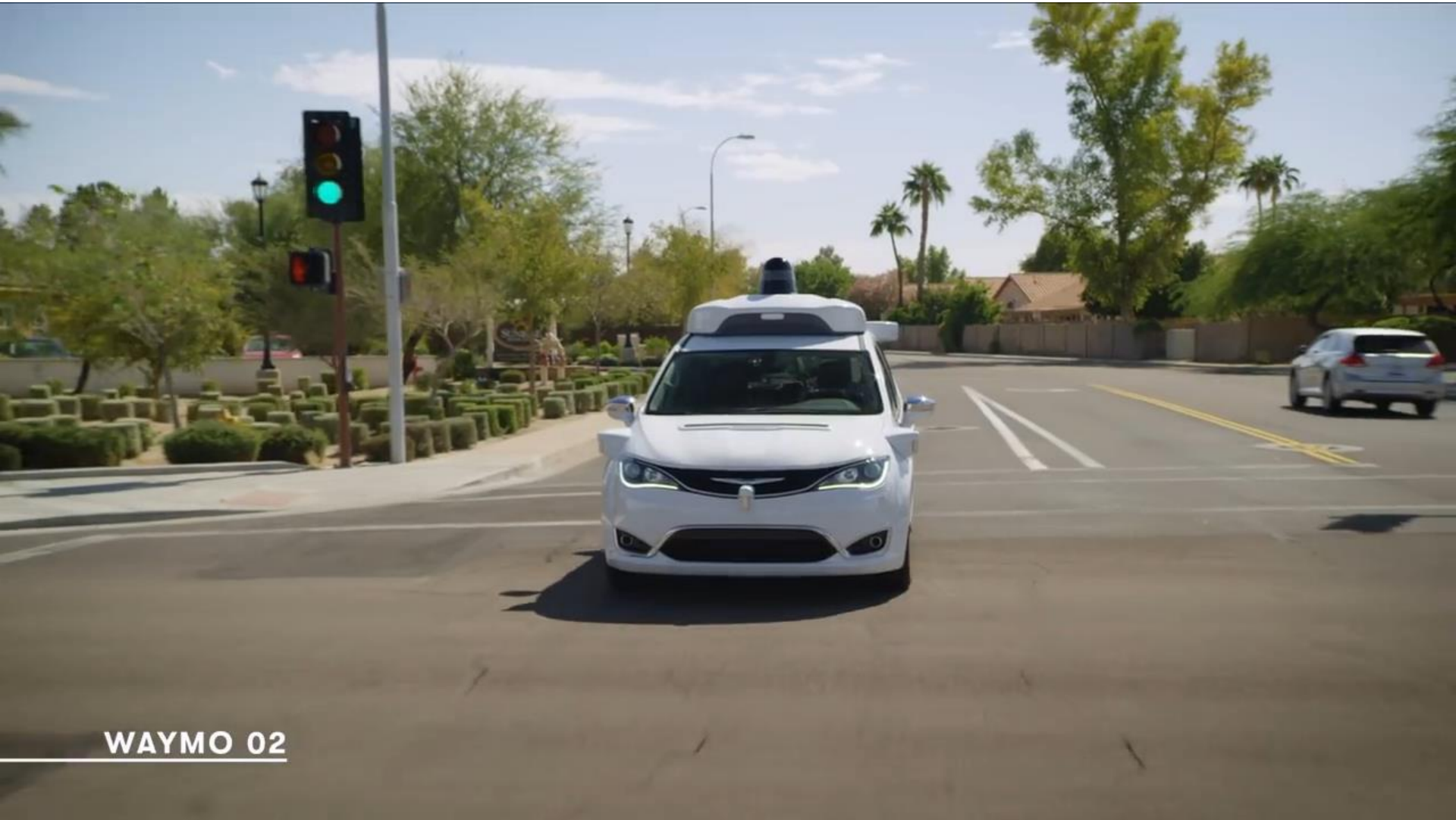


AlphaZero vs. AGO



AZ wins AZ draws AZ loses AZ white AZ black

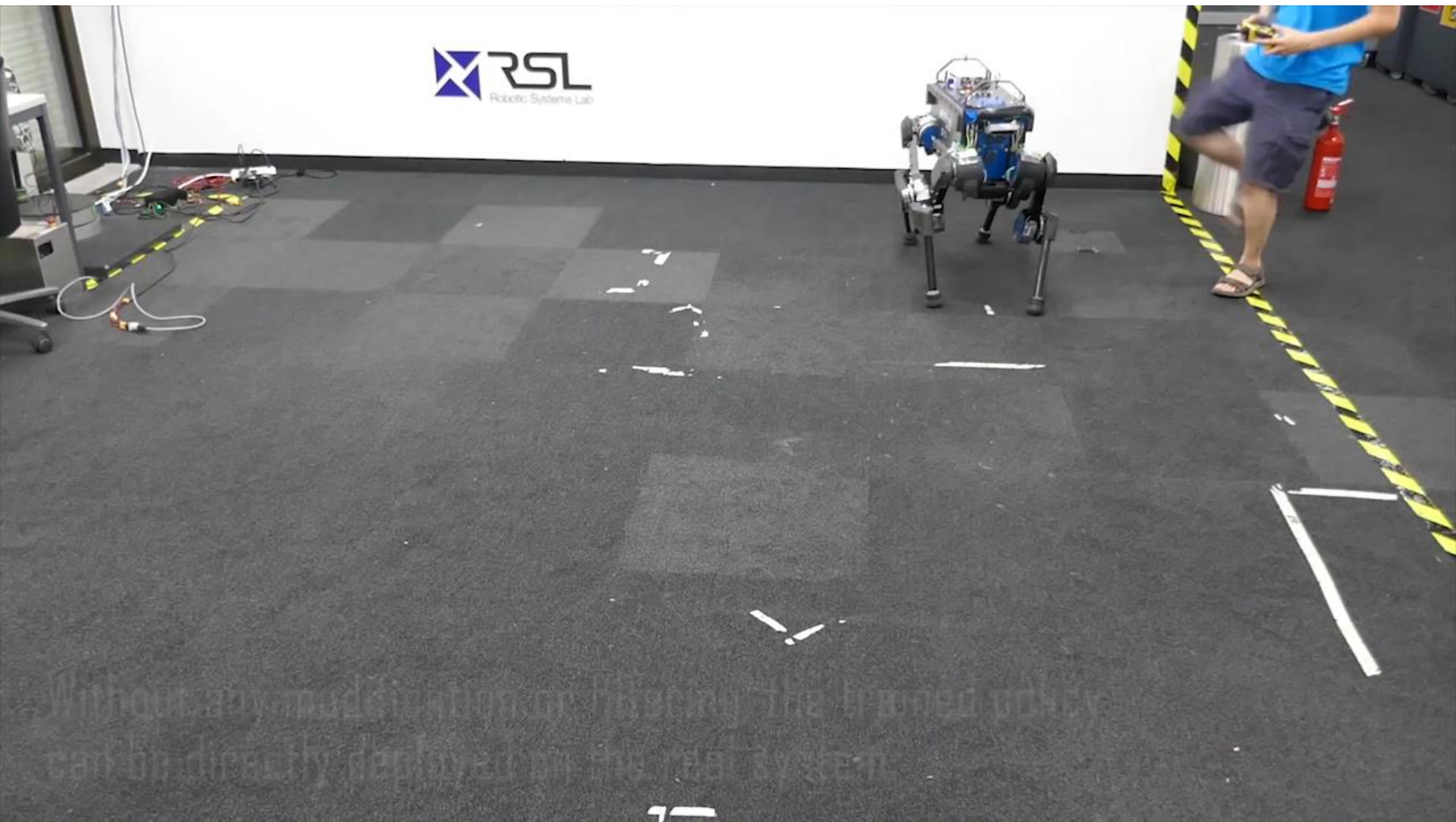
To date, for **most** successful robots operating in the real world:
Deep RL is not involved



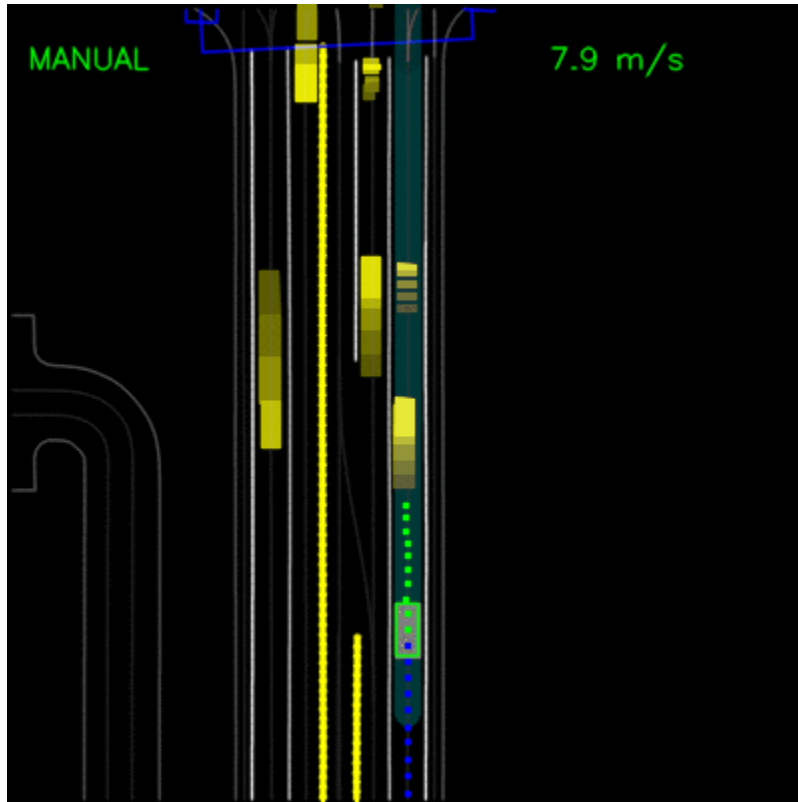
To date, for **most** successful robots operating in the real world:
Deep RL is not involved



But... that's slowly changing: Learning Control Dynamics



But... that's slowly changing:
Learning to Drive: Beyond Pure Imitation (Waymo)



The Challenge for RL in Real-World Applications

Reminder:

Supervised learning:
teach by example

Reinforcement learning:
teach by experience

Open Challenges. Two Options:

1. Real world observation + one-shot trial & error
2. Realistic simulation + transfer learning

